arXiv:2502.05591v5 [cs.DC] 1 Jul 2026

Round and Resilience-Optimal Approximate Agreement on Trees and Block Graphs

Marc Fuchs [1], Diana Ghinea [2], Zahra Parsaeian [1], and Joel Rybicki [3]

[1]University of Freiburg, {marc.fuchs,zahra.parsaeian}@cs.uni-freiburg.de
[2]Lucerne University of Applied Sciences and Arts, diana.ghinea@hslu.ch
[3]Humboldt University of Berlin, joel.rybicki@hu-berlin.de

Abstract

Approximate Agreement ($\mathcal{AA}$) is a fundamental primitive that, even in the presence of Byzantine faults, allows honest parties to obtain close (but not necessarily identical) outputs that lie within the range of their inputs. While the optimal round complexity of synchronous $\mathcal{AA}$ on real values is well understood, its extension to other input spaces has remained open, with fundamental questions regarding achievable resilience and round efficiency still unresolved.

In this work, we investigate the optimal round complexity of synchronous $\mathcal{AA}$ on trees under Byzantine failures. In this setting, parties hold as inputs vertices of a publicly known labeled tree T and must output 1-close vertices lying in the convex hull of the honest inputs. We present a synchronous protocol with round complexity $O\left(\frac{\log \mathsf{D}(T)}{\log\log \mathsf{D}(T)}\right)$, where $\mathsf{D}(T)$ denotes the diameter of the input space tree. Our protocol is obtained via a reduction to real-valued $\mathcal{AA}$, and maintains the resilience guarantees of the underlying real-valued $\mathcal{AA}$ protocol: $t = c \cdot n$, for any constant $c < 1/3$ in settings with no cryptographic assumptions, and for any constant $c \in [1/3, 1/2)$ assuming digital signatures. Complementing this result, we extend impossibility results for real-valued $\mathcal{AA}$ to any graph G by proving a lower bound of $\Omega\left(\frac{\log \mathsf{D}(G)}{\log\log \mathsf{D}(G)+\log \frac{n+t}{t}}\right)$ rounds, where n is the number of parties and t the number of Byzantine faults. Together, these results establish the asymptotic optimality of our protocol whenever $t \in \Theta(n)$.

We further extend our techniques to block graphs by leveraging their clique tree structure. This yields protocols for $\mathcal{AA}$ on block graphs with optimal resilience in both the synchronous and asynchronous models; in the synchronous setting, the resulting protocols achieve asymptotically optimal round complexity. In doing so, we make progress on a broader open problem: beyond trees, the optimal resilience thresholds for $\mathcal{AA}$ on graph input spaces were largely unresolved, and our results pin down these thresholds for the class of block graphs. For the exact-agreement variant of the problem, the resilience threshold has been shown to be $t < n/\omega$ on any graph with clique number ω, where t denotes the number of Byzantine faults. Our work shows that a similar lower bound does not hold in general for $\mathcal{AA}$.

Finally, in contrast to prior protocols for $\mathcal{AA}$ beyond the real line, which typically rely on safe-area constructions and can incur exponential local computation, our approach in the synchronous model reduces the problem to real-valued $\mathcal{AA}$ and therefore admits efficient local computation.

1 Introduction

Ensuring consistency among parties in a distributed system is essential, yet it is a difficult task in the face of potential failures or malicious behavior. Agreement protocols serve as indispensable tools for achieving consensus in such environments. One such fundamental primitive is Approximate Agreement ($\mathcal{AA}$) [15]. The Byzantine variant of this problem considers a setting of n parties in a network, where each party holds a real value as input. Even when t out of the n parties involved are Byzantine (i.e., malicious), $\mathcal{AA}$ enables the honest parties to obtain ε-*close* values (for any predefined error $\varepsilon > 0$) that lie within the range spanned by the honest inputs. This relaxed form of agreement has proven useful in scenarios where exact consensus is either unnecessary or infeasible, such as clock synchronization [30], blockchain oracles [6], distributed machine learning [17,18,40], aviation control systems [32,39], and robot gathering [38].

The $\mathcal{AA}$ problem is not limited to real-valued inputs. Several variants have been explored, including multidimensional real inputs [33, 34, 41] and discrete domains such as various classes of (connected) graphs [2, 5, 13, 31, 37]. These generalizations naturally extend the original requirements: the honest outputs still need to be close and they must be in the *convex hull* of the honest inputs (defined according to the convexity notion associated with the input space). Intuitively, this means that honest outputs must lie "inside the region" spanned by the honest inputs; formal definitions will be provided later.

Regardless of the input space considered, the $\mathcal{AA}$ problem admits solutions that follow a common iteration-based outline. In each iteration, the parties use a mechanism to distribute their current values to all parties. Based on the values received, each party computes a *safe area*: this is a set guaranteed to lie within the convex hull of the values distributed by the honest parties. The honest parties' safe areas are not necessarily identical, but they have some overlap. Each party then computes its new value essentially as a midpoint of the safe area it has obtained. The parties' new values remain within the honest inputs' convex hull, and they become closer with each iteration. This way, after a sufficient number of iterations, $\mathcal{AA}$ is achieved.

While this outline has been a powerful tool in establishing sufficient conditions for various input spaces, multiple questions remain open: both in terms of resilience and efficiency (e.g., rounds, communication, local computation). Except for the communication complexity of short inputs, $\mathcal{AA}$ on real values is well understood in the synchronous model [7, 19, 20, 24, 28, 36]. Moreover, while optimal resilience thresholds are known for spaces such as $\mathbb{R}^d$ [34] and trees [13, 37], a unified understanding of optimal efficiency tradeoffs, as well as resilience in more general graph classes, remains elusive.

This paper makes a step forward by exploring optimal round complexity for $\mathcal{AA}$ under Byzantine faults beyond the real line, namely on trees, in the synchronous model. In this variant [37], the input space is a publicly-known labeled tree T. Each party holds a vertex of T as input, and every honest party must output a vertex of T such that the following conditions hold: (i) honest outputs must be within distance one of each other, and (ii) honest outputs must lie in the honest inputs' convex hull (in the smallest subtree of T containing all honest input vertices).

1.1 Our Contribution

Results for trees. We establish tight asymptotic round-complexity bounds for synchronous $\mathcal{AA}$ on trees under Byzantine faults. We first show that $\Omega\left(\frac{\log \mathsf{D}(G)}{\log\log \mathsf{D}(G)+\log \frac{n+t}{t}}\right)$ rounds are necessary for $\mathcal{AA}$ on any graph G, where $\mathsf{D}(G)$ denotes the diameter of G. This result is obtained by adapting Fekete's bound [20] from $\mathcal{AA}$ on real numbers. Afterwards, we show that this lower bound is tight for $\mathcal{AA}$ on a tree whenever $t \in \Theta(n)$ by reducing the problem to real-valued $\mathcal{AA}$: we achieve $\mathcal{AA}$ on any tree T with round complexity of $O\left(\frac{\log \mathsf{D}(T)}{\log\log \mathsf{D}(T)}\right)$ by using a round-optimal real-valued $\mathcal{AA}$ protocol as a building block. At the same time, our protocol achieves optimal resilience: depending on the real-valued $\mathcal{AA}$ protocol used, our protocol is resilient up to $t < n/3$ Byzantine parties without cryptographic assumptions, and

up to $t = c \cdot n$ Byzantine parties, for any constant $c \in [1/3, 1/2)$, given a public-key infrastructure and a secure digital signature scheme.

Extension beyond trees: block graphs. We show that our techniques extend also to a more general class of graphs, namely block graphs. Block graphs admit a clique tree in which adjacent cliques intersect in exactly one vertex, which enables us to reduce $\mathcal{AA}$ on any connected block graph G to $\mathcal{AA}$ on a tree T. Such clique tree-based reductions have appeared before in the wait-free setting (i.e., asynchronous shared memory, with up to $t < n$ crashes) with weaker validity conditions [2]. Using our protocol for trees, we get a synchronous protocol with asymptotically optimal round complexity $O\left(\frac{\log \mathsf{D}(G)}{\log \log \mathsf{D}(G)}\right)$ whenever $t \in \Theta(n)$. The resilience guarantees are inherited from the underlying real-valued $\mathcal{AA}$ protocol: $t < n/3$ Byzantine parties without cryptographic assumptions, and up to $t = c \cdot n$ Byzantine parties, where $c \in [1/3, 1/2)$ is a constant, given a public-key infrastructure and a secure digital signature scheme. Moreover, if we instead use the asynchronous protocol for trees of [37] instead of our synchronous protocol, we get an asynchronous protocol for G with optimal resilience $t < n/3$.

In comparison to prior work, we observe the following:

- Our results improve over previously known sufficient conditions for achieving $\mathcal{AA}$ on block graphs of maximum clique size ω: $t < n/\omega$ in the synchronous model, and $t < n/(\omega + 1)$ in the asynchronous model [13, 37]. These conditions were only known to be tight for the exact-agreement variant of the problem, known as Convex Agreement ($\mathcal{CA}$). As $\mathcal{CA}$ and $\mathcal{AA}$ have matching optimal resilience thresholds for $\mathbb{R}, \mathbb{R}^d$ and trees, it may be natural to assume that the optimal resilience thresholds match for other input spaces as well. Our result for block graphs shows that this is not the case, and leaves the following question open: characterizing the boundary at which $\mathcal{AA}$ and $\mathcal{CA}$ separate in resilience.

- Previous protocols achieving $\mathcal{AA}$ on spaces beyond the real line involve high local computation complexity due to the safe-area approach, which requires each party to intersect up to $\binom{n}{n-t}$ convex sets in each round [13, 26, 33, 34, 37, 41]. Our synchronous protocol for block graphs avoids this overhead – without relaxing the requirements on the honest outputs, by reducing the problem to real-valued $\mathcal{AA}$, the per-round local computation comes down to sorting up to n real values, discarding the outliers (i.e., the lowest t and the highest t) and computing the average of the remaining values. It would be interesting to see if the safe area computation can be avoided for other input spaces.

1.2 Related work

Real values. $\mathcal{AA}$ was first introduced by Dolev et al. [15]. They investigated the feasibility of achieving $\mathcal{AA}$ on $\mathbb{R}$, both in the synchronous and the asynchronous model. Concretely, Dolev et al. [15] proposed a synchronous protocol resilient against $t < n/3$ corruptions, which is optimal in settings with no cryptographic assumptions, and an asynchronous protocol resilient against $t < n/5$ corruptions. The resilience condition $t < n/3$ was proven to be optimal in the asynchronous model as well as in [1, 12], and the condition $t < n/2$ was proven to be tight in the synchronous model with cryptographic assumptions [25, 30]. Real-valued $\mathcal{AA}$ has also been completely characterized in terms of resilience in the *network-agnostic model* in [25]: a network-agnostic protocol must tolerate up to t_s corruptions if it runs in a synchronous network, and up to $t_a \leq t_s$ corruptions if it runs in an asynchronous network (without knowing the type of network a priori).

These protocols follow the aforementioned iteration-based outline and, given that the honest inputs are D-close, incur round complexity of roughly $O(\log(\mathsf{D}/\varepsilon))$. The work of [15] showed that this is optimal for a specific class of protocols that do not leverage information obtained in prior iterations. Fekete [19, 20] investigated this further and provided lower bounds on *how close the honest values may get* after

R rounds, considering various types of failures (crash, omission, Byzantine) in both synchronous and asynchronous models. In the synchronous model with Byzantine failures, [20] shows that any R-round protocol has an execution where the range of honest values is reduced by a (multiplicative) factor of at least $\frac{t^R}{R^R \cdot (n+t)^R}$ – in contrast to the 2^{-R} factor obtained by prior R-iteration $\mathcal{AA}$ protocols. This lower bound is asymptotically tight: [20] has also provided an asymptotically matching protocol assuming $t < n/4$, with message complexity $O(n^R)$. Later, [7] improved the resilience threshold to $t < n/3$ and reduced the message complexity to $O(R \cdot n^3)$. In addition, with minor adjustments, the *Proxcensus* protocol of [23] becomes an $\mathcal{AA}$ protocol that matches Fekete's lower bound for $t = c \cdot n$ corruptions, for any constant $c < 1/2$, assuming digital signatures. For the asynchronous model, the lower bound of [19] holds for protocols in canonical-round form – as discussed in [4], this does not cover protocols such as [1, 12].

Optimizations regarding communication complexity have also been a topic of interest. The work of [36] achieves optimal-resilience asynchronous $\mathcal{AA}$ with $O(n^2)$ messages per iteration, in contrast to the $O(n^3)$ messages required by earlier solutions [1]. In addition, [27, 28] show that $O(\ell n)$ bits of communication are sufficient to achieve a stronger variant of $\mathcal{CA}$ on ℓ-bit integer inputs, provided that ℓ is sufficiently large, and [24] extended this result to real-valued $\mathcal{AA}$.

Trees and graphs. The problem of approximately agreeing on vertices in a graph was first considered by Alcántara et al. [2] and Nowak and Rybicki [37]. Alcántara et al. [2] introduced two variants: *Edge-Gathering*, where honest parties output vertices that are either identical or adjacent, and *1-Gathering*, where the honest outputs form a clique. When the input space is a tree, these two variants are equivalent. These variants were presented in the wait-free model, framed as relaxations of the *robot gathering* problem [14, 21]. It is important to note that these formulations differ from the classical $\mathcal{AA}$ requirements in that they do not require the outputs to lie within the convex hull of honest inputs (unless the honest inputs already lie on the same edge in the case of Edge-Gathering, or on a clique in the case of 1-Gathering).

Nowak and Rybicki [37] generalized the $\mathcal{AA}$ problem to arbitrary convexity spaces in a fully-connected network with Byzantine failures. They extended the problem to settings such as trees and chordal graphs, and proposed protocols that operate in the asynchronous model. In addition, their work provides a characterization of the exact agreement version of $\mathcal{AA}$ on arbitrary convex spaces in the synchronous model. These problems were later explored in the network-agnostic model by [13]. Regarding round complexity, the asynchronous $\mathcal{AA}$ protocol of [37] for trees achieves round complexity $O(\log \mathsf{D})$, where D is the diameter of the input space tree. While further work focused on improving the message complexity [36], the round complexity of $O(\log \mathsf{D})$ remains the state of the art in the asynchronous model, as well as in the synchronous model (up to our work). For chordal graphs, [37] proposed two protocols: one presented as a protocol for cycle-free semilattices (i.e., a particular case of chordal graphs) with round complexity $O(\mathsf{D})$, and a more efficient protocol with $O(\log \mathsf{D})$ rounds, where D denotes the input space diameter of the graph. However, the latter protocol used a reduction to tree decompositions of the chordal graph [37] and was shown to be incorrect by [13], which provided a network-agnostic protocol with $O(\mathsf{D})$ rounds. Whether $o(\mathsf{D})$ rounds are, in fact, sufficient for chordal graphs has remained an open problem; our work resolves this in the affirmative for the subclass of block graphs.

A central question regarding $\mathcal{AA}$ on graphs is characterizing the classes of graphs that admit $\mathcal{AA}$ protocols. While the protocols of [13, 37] assume that the graph is chordal, multiple works have explored structural properties required for $\mathcal{AA}$ on graphs in the wait-free model [2, 3, 29, 31].

2 Preliminaries

We describe the model and key concepts and definitions.

Model and Adversary. We consider n parties $p_1, p_2, \ldots, p_n$ running a protocol in a fully-connected network where links model authenticated channels. We assume that the network is synchronous: the parties' clocks are synchronized and every message is delivered within a publicly known amount of time

Δ. Our protocols assume a (computationally unbounded) adversary that can permanently corrupt up to $t < n/3$ parties. The adversary is adaptive – it may choose which parties to corrupt at any point in the protocol's execution. Corrupted parties become Byzantine: they may deviate arbitrarily (maliciously) from the protocol. Our lower bound, however, holds even against a static adversary that has to choose which parties to corrupt at the beginning of the protocol's execution.

Approximate Agreement on $\mathbb{R}$. We recall the definition of $\mathcal{AA}$ on real values, as presented in [15].

Definition 1. *Let $\Pi(\varepsilon)$ be an n-party protocol in which every party holds a value in $\mathbb{R}$ as input and ε is a publicly known parameter. We say that $\Pi(\varepsilon)$ achieves $\mathcal{AA}$ if the following properties hold for any predefined $\varepsilon > 0$ even when up to t of the n parties are corrupted:* ***(Termination)*** *Every honest party produces an output in $\mathbb{R}$ and terminates;* ***(Validity)*** *Honest parties' outputs lie within the range of the honest inputs;* ***(ε-Agreement)*** *If two honest parties output v and v', then $|v - v'| \leq \varepsilon$.*

We use the term *valid value* to refer to a value within the range of the honest inputs.

Notations for graphs. For a graph $G = (V, E)$, we use $\mathsf{V}(G)$ to denote its set of vertices. We denote the shortest path between two vertices $u, v \in \mathsf{V}(G)$ by $P(u, v)$: as we are concerned with trees and block graphs, these are unique. We define the distance $\mathsf{d}(u, v)$ as the length of $P(u, v)$. We use $\mathsf{D}(G)$ to denote the diameter of G, i.e., the maximum distance between any two vertices in $\mathsf{V}(G)$. The *(shortest path) convex hull* $\langle S \rangle$ of a set of vertices $S \subseteq \mathsf{V}(G)$ is the union of all shortest paths between any pair of vertices in S. Note that for trees, this is equivalent to the set of vertices in the smallest connected subtree containing S.

$\mathcal{AA}$ on a graph G. We now define the problem of $\mathcal{AA}$ on graph G [37].

Definition 2. *Let G be a fixed labeled graph known to all parties and $\Pi(G)$ be an n-party protocol in which every party holds a vertex of G as input. We say that $\Pi(G)$ achieves $\mathcal{AA}$ on G if the following properties hold even when up to t of the n parties are corrupted:* ***(Termination)*** *Every honest party produces an output in $\mathsf{V}(G)$ and terminates;* ***(Validity)*** *Honest parties' outputs lie within the honest inputs' convex hull;* ***(1-Agreement)*** *If two honest parties output v and v', then $\mathsf{d}(v, v') \leq 1$.*

As in the case of real values, we refer to a vertex in the honest inputs' convex hull as a *valid vertex*. If the input space graph G has diameter $\mathsf{D}(G) \leq 1$ (i.e., it is a single clique), the $\mathcal{AA}$ problem becomes trivial: each party can simply return its own input. Therefore, in the remainder of this work, we focus on graphs with diameter $\mathsf{D}(G) > 1$. We add that, except for the case where G is a tree, this does not apply to the *Edge-Agreement* variant of [2], where the honest parties' outputs need to be on the same edge.

Round complexity. In a synchronous communication round, every party may send messages to the other parties, receive all messages sent to it during that round, and perform local computations. The round complexity of a protocol is the maximum number of such rounds until every honest party terminates, over all admissible executions.

3 Lower Bound

In this section, we analyze the round complexity required to achieve $\mathcal{AA}$ on a graph. We begin by revisiting Fekete's lower bound for real-valued $\mathcal{AA}$ regarding *how close the honest values may get* after a fixed number of rounds. Fekete [20] used chain arguments to prove lower bounds for the convergence rate of real-valued $\mathcal{AA}$ protocols. However, upon closer inspection, his proof only relies on a weaker validity condition than Definition 1 called *Strong Unanimity*: if all honest parties hold input v, no honest party outputs $v' \neq v$.

Fekete constructed for the R-round full-information protocol a chain of $s \leq \frac{(n+t)^R}{t_1 \cdot t_2 \cdot \cdots \cdot t_R}$ views, where $\sum_{i=1}^{R} t_i \leq t$, and in the first view, all honest parties start with input a, and in the last view, all honest

parties start with input b. For these views, t_i roughly represents the number of Byzantine parties that deviate from the protocol for the first time in round i. For any pair of views that are consecutive in the chain, there exists an execution of the full-information protocol in which two honest parties obtain those views.

For any protocol satisfying strong unanimity, the first view leads to all honest parties outputting a and the last view leads to all honest parties outputting b. In the case of real values, this implies the existence of two consecutive views in the chain that yield two honest outputs $v, v' \in \mathbb{R}$ with $|v - v'| \geq |a - b|/s$. However, the proof works as-is on any set V of values with a distance metric d. Following the exact same steps as in Fekete's proof [20, Theorem 15], we get the next lemma.

Lemma 1. *Let Π be any deterministic R-round protocol with inputs and outputs from the set V that satisfies strong unanimity even when up to t of the n parties involved are Byzantine. Then for any $a, b \in V$, there exists an execution of Π in which two honest parties output v and v' such that $\mathsf{d}(v, v') \geq \mathsf{d}(a, b) \cdot \left(\frac{t}{R(n+t)}\right)^R$.*

The lower bound is an immediate corollary of the above lemma. By Fekete's results [20, Theorem 18], the same asymptotic lower bound holds for crash faults by replacing $t/(n+t)$ with $t/(2n+3t)$.

Theorem 1. *Let $n > t > 0$ be fixed. Any deterministic n-party protocol that achieves $\mathcal{AA}$ on a graph G even when up to t of the n parties are Byzantine requires $\Omega\left(\frac{\log \mathsf{D}(G)}{\log\log \mathsf{D}(G) + \log \frac{n+t}{t}}\right)$ rounds to terminate.*

Proof. Let $\delta := (n+t)/t$. Choose $a, b \in V$ such that $\mathsf{d}(a, b) = \mathsf{D}(G)$. Then by 1-Agreement and Lemma 1, any honest parties' output values v and v' satisfy

$$1 \geq \mathsf{d}(v, v') \geq \mathsf{D}(G) \cdot \left(\frac{t}{R(n+t)}\right)^R = \mathsf{D}(G) \cdot \left(\frac{1}{R\delta}\right)^R.$$

By rearranging the above, the claim now follows for any fixed δ, as

$$\begin{aligned} (R\delta)^R \geq \mathsf{D}(G) &\iff R\log(R\delta) \geq \log \mathsf{D}(G) \\ &\iff R \geq \Omega\left(\frac{\log \mathsf{D}(G)}{\log\log \mathsf{D}(G) + \log \delta}\right). \end{aligned}$$

□

4 Round-Optimal Protocol for Trees

In this section, we give our round-optimal $\mathcal{AA}$ protocol for labeled trees, asymptotically matching the lower bound of Section 3. We begin with the special case where the input space is a path, where the problem reduces cleanly to real-valued $\mathcal{AA}$. We then lift this warm-up to general trees by projecting inputs onto suitably chosen paths.

4.1 Warm-up: Protocol for Paths

Building towards our solution, we begin by describing a warm-up protocol for the case where the input space is a labeled path P. We make use of the protocol of [7], denoted by RealAA, which achieves $\mathcal{AA}$ on $\mathbb{R}$ with asymptotically optimal round complexity, described by the theorem below. We add that the round complexity analysis presented in [7] assumes $\varepsilon = 1/n$. We extend their analysis to any $\varepsilon > 0$. The technical details of the proof are included in Section B.

Theorem 2. *There is a protocol* $\mathsf{RealAA}(\varepsilon)$ *achieving* $\mathcal{AA}$ *on real values even when up to* $t < n/3$ *of the* n *parties involved are Byzantine. If the honest inputs are* D*-close,* $\mathsf{RealAA}(\varepsilon)$ *ensures Termination within* $R_{\mathsf{RealAA}}(\mathsf{D}, \varepsilon) < 7 \cdot \frac{\log_2(\mathsf{D}/\varepsilon)}{\log_2 \log_2(\mathsf{D}/\varepsilon)} + 3$ *rounds.*

To achieve $\mathcal{AA}$ on a path P, the parties denote the $k := \mathsf{D}(P)$ vertices in the path P by $(v_1, v_2, \ldots, v_k)$ (where v_i, v_{i+1} are adjacent, and v_1 is the endpoint of P with the lowest label in lexicographic order). A party with input v_i joins $\mathsf{RealAA}(1)$ with input i. Each party obtains a real value j from $\mathsf{RealAA}(1)$ and outputs the vertex $v_{\lfloor j \rceil}$, where $\lfloor j \rceil$ denotes the nearest integer to j: if $z \leq j < z + 1$ for $z \in \mathbb{Z}$, $\lfloor j \rceil := z$ if $j - z < (z + 1) - j$ and $\lfloor j \rceil := z + 1$ otherwise. The following properties are easy to show.

Remark 1.

1. *If* $i_{\min}, i_{\max} \in \mathbb{Z}$ *and* $i_{\min} \leq j \leq i_{\max}$*, then* $i_{\min} \leq \lfloor j \rceil \leq i_{\max}$*.*
2. *If* $j, j' \in \mathbb{R}$ *satisfy* $\left|j - j'\right| \leq 1$*, then* $\left|\lfloor j \rceil - \lfloor j' \rceil\right| \leq 1$*.*

The first part of Remark 1 follows directly from the definition of $\lfloor \cdot \rceil$ and ensures that the vertices $v_{\lfloor j \rceil}$ obtained are valid. For 1-Agreement, $\mathsf{RealAA}(1)$ provides the honest parties with 1-close real values j. The second part of Remark 1 ensures that the honest parties hold 1-close integer values $\lfloor j \rceil$. Hence, the honest parties' output vertices $v_{\lfloor j \rceil}$ are 1-close, and 1-Agreement holds. Consequently, we have achieved $\mathcal{AA}$ on the path P within $O\left(\frac{\log \mathsf{D}(P)}{\log \log \mathsf{D}(P)}\right)$ rounds by Theorem 2.

4.2 Moving Towards Trees

As a step towards our protocol for trees, we assume that the parties *know* a path P in the input space tree T that intersects the honest inputs' convex hull. Then, the parties may proceed as follows: each party with input $v_{\text{IN}} \in \mathsf{V}(T)$ computes the projection of the vertex v_{IN} onto the path P, denoted by $\mathsf{proj}_P(v_{\text{IN}})$. This is the vertex in P that has the shortest distance to v_{IN}, as shown in Figure 1. Note that the honest parties' projections $\mathsf{proj}_P(v_{\text{IN}})$ are in the honest inputs' convex hull, as described by the lemma below. The proof is included in Section A.

Lemma 2. *Consider a set of vertices* $S \subseteq \mathsf{V}(T)$ *and a path* P *in* T *such that* $\mathsf{V}(P) \cap \langle S \rangle \neq \varnothing$*. Then, for any* $v \in S$*,* $\mathsf{proj}_P(v) \in \mathsf{V}(P) \cap \langle S \rangle$*.*

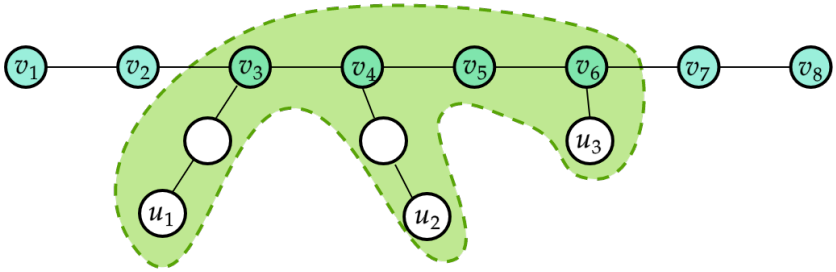


Figure 1: Let P be the assumed path, represented by the sequence of vertices $v_1, v_2, \ldots, v_8$. The vertices u_1, u_2, u_3 correspond to the honest inputs, whose convex hull is highlighted in green. The projections of u_1, u_2, u_3 onto path P are vertices v_3, v_4, v_6, respectively.

Then, the parties follow the approach described in Section 4.1 to reach $\mathcal{AA}$ on the path P, using the vertices $\mathsf{proj}_P(v_{\text{IN}})$ as inputs. This provides the parties with vertices that are 1-close in T, and by Lemma 2, valid in T. Consequently, $\mathcal{AA}$ is achieved within $O\left(\frac{\log \mathsf{D}(T)}{\log \log \mathsf{D}(T)}\right)$ rounds.

4.3 Weakening the Path Assumption: Initial Attempt

In Section 4.2, we have discussed a round-optimal approach for $\mathcal{AA}$ on trees given that the honest parties *know* a path in T that intersects the honest inputs' convex hull. This section makes a first step towards removing this assumption, leading to a solution with round complexity $O\left(\frac{\log\left|\mathsf{V}(T)\right|}{\log\log\left|\mathsf{V}(T)\right|}\right)$. This is the protocol we have described in [22], and represents a stepping stone towards our final solution for trees, presented in Section 4.4.

Intuitively, it may seem that finding a path P that passes through the honest inputs' convex hull comes down to solving *Byzantine Agreement*. This would require $t+1 = O(n)$ communication rounds [16], which generally prevents us from achieving our round complexity goal. Instead, we implement a subprotocol $\mathsf{PathsFinder}^0$ that enables the honest parties to *approximately* agree on such a path. Concretely, each honest party obtains a subpath P of T such that: (i) P intersects the convex hull of the honest inputs; and (ii) if two honest parties obtain different paths P and P', then either P extends P' by one edge, or P' extends P by one edge. Formally, if $P = (v_1, \ldots, v_k)$ and $P' = (u_1, \ldots, u_{k'})$, then either $P = P'$, or $P' = P \oplus (v_k, u_{k'})$, or $P = P' \oplus (u_{k'}, v_k)$. In Section 4.3.2, we show that this suffices to apply the approach from Section 4.2.

4.3.1 Approximately Agreeing on Paths

To obtain these paths, we assume that the input space tree T is rooted, and we denote its root by v_{root} (this will be the vertex in T with the lowest label in lexicographic order). We then enable the honest parties to achieve 1-Agreement on vertices within a subtree rooted at a valid vertex. Once an honest party obtains such a vertex v, it is guaranteed that the path $P(v_{\mathsf{root}}, v)$ intersects the convex hull of the honest inputs. Moreover, the 1-Agreement property ensures that the honest parties' paths $P(v_{\mathsf{root}}, v)$ are identical except possibly for one additional edge.

List representation. Our approach for enabling honest parties to identify vertices within a subtree rooted at a valid vertex is based on transforming the (now rooted) tree T into a list L with certain key properties. This transformation relies on a technique commonly used for efficiently computing *lowest common ancestors* in rooted trees [9]. Each party performs a depth-first search (DFS) starting from the fixed root vertex v_{root} and records each vertex upon every visit during the DFS traversal. We present the code for this step below. Afterwards, we describe the list obtained in this step for the tree depicted in Figure 2.

$\mathsf{ListConstruction}(T, v_{\mathsf{root}})$

DFS(v), where $v \in \mathsf{V}(T)$, L is a global list:

1: Append v to L.
2: For each neighbor $v' \notin L$ of v (ordered by labels):
3: Run DFS(v'). Afterwards, append v to L.

Code for party p, given tree T and a vertex $v_{\mathsf{root}} \in \mathsf{V}(T)$:

1: $L := [\,]$. Run DFS(v_{root}). Afterwards, **return** L.

Figure 2 illustrates an example of an input space tree. If $v_{\mathsf{root}} := v_1$, every party begins the DFS from v_1. The traversal then visits v_2, proceeds to v_3, then v_6, returns to v_3, continues to v_7, returns again to v_3, then to v_2, and so on. The final list obtained from this traversal is $L = [v_1, v_2, v_3, v_6, v_3, v_7, v_3, v_2, v_4, v_8, v_4, v_2, v_5, v_2, v_1]$.

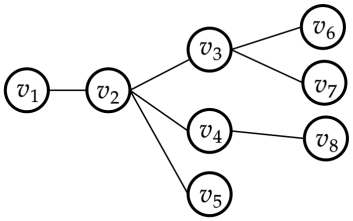


Figure 2: An input space tree.

To describe the properties of the resulting list, we first establish a few notations. We use L_i (for $1 \leq i \leq \left|L\right|$) to denote the i-th element

of list L. In addition, for a vertex $v \in \mathsf{V}(T)$, we let $L(v)$ denote the set of indices i such that $L_i = v$.

Lemma 3 formally establishes the guarantees of the list constructed via this approach. The proof is provided in Appendix E.

Lemma 3. *Consider a rooted tree T, and let v_{root} denote its root.*

Then, $\mathsf{ListConstruction}(T, v_{\mathsf{root}})$ returns, in finite time, a list of vertices L with the following properties:

1. *If $\left|\mathsf{V}(T)\right| > 1$, then for any $i < \left|L\right|$, the vertices L_i and L_{i+1} are adjacent in T.*
2. *The list L contains $\left|L\right| \leq 2 \cdot \left|\mathsf{V}(T)\right|$ elements, and, for every vertex $v \in \mathsf{V}(T)$, we have $L(v) \neq \varnothing$.*
3. *Consider a vertex $v \in \mathsf{V}(T)$, and let $i_{\min} = \min L(v)$ and $i_{\max} = \max L(v)$. Then, a vertex u is in the subtree rooted at v if and only if $L(u) \subseteq [i_{\min}, i_{\max}]$.*
4. *For any two vertices $v, v' \in \mathsf{V}(T)$ and any $i \in L(v)$ and $i' \in L(v')$, the lowest common ancestor of v and v' is in the set $\{L_k : \min(i, i') \leq k \leq \max(i, i')\}$.*

Note that the ListConstruction algorithm is deterministic, and therefore enables the honest parties to (simultaneously) obtain the same list L.

Finding vertices in the subtree of a valid vertex. After computing the list L, each party p with input vertex v_{IN} joins $\mathsf{RealAA}(1)$ with input $i \in L(v_{\text{IN}})$. We add that $L(v_{\text{IN}})$ may contain multiple indices, and the parties may choose any of them; without loss of generality, we let $i := \min\{L(v_{\text{IN}})\}$.

Then, $\mathsf{RealAA}(1)$ provides the parties with 1-close real values j that lie within the range of honest values i. By applying Remark 1, we conclude that the honest parties obtain 1-close integer values $\lfloor j \rfloor$ that are within the range of honest values i. Finally, using Lemma 3, it follows that the list elements $L_{\lfloor j \rfloor}$ corresponding to honest parties' values j are 1-close vertices that belong to a subtree rooted at a valid vertex.

We note that this **does not** imply that the vertices $L_{\lfloor j \rfloor}$ are valid. Consider again the input space tree in Figure 2, depicted in Figure 3. If the honest inputs are v_3, v_6, and v_5, their convex hull is $\{v_5, v_2, v_3, v_6\}$. The honest parties join $\mathsf{RealAA}(1)$ with indices i chosen from $L(v_3) = \{3, 5, 7\}$, $L(v_6) = \{4\}$, and $L(v_5) = \{13\}$. Note that the indices in $L(v_4) = \{9, 11\}$ and $L(v_8) = \{10\}$ lie within the range of honest indices i. Thus, $\mathsf{RealAA}(1)$ may return outputs j such that $\lfloor j \rfloor$ corresponds to, for example, v_4 or v_8 – these are vertices that lie outside the honest inputs' convex hull $\{v_5, v_2, v_3, v_6\}$. However, v_4 and v_8 still belong to the subtree rooted at the valid vertex v_2.

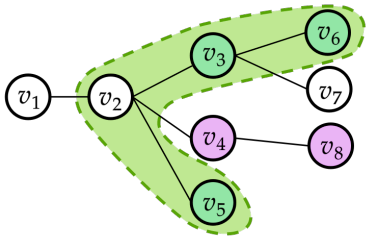


Figure 3: Vertices v_4, v_8 are not valid, but are in the subtree of a valid vertex (with respect to root v_1).

Paths intersecting the honest inputs' convex hull. In the example above, since vertices v_4 and v_8 lie in the subtree rooted at the valid vertex v_2, the paths $P(v_1, v_4)$ and $P(v_1, v_8)$ intersect the convex hull of the honest inputs. The lemma below shows that this property holds more generally: if an honest party obtains vertex $L_{\lfloor j \rfloor}$ via our approach, then the path $P(v_{\mathsf{root}}, L_{\lfloor j \rfloor})$ intersects the honest inputs' convex hull.

Lemma 4. *Assume a rooted tree T with root v_{root} and let $L := \mathsf{ListConstruction}(T, v_{\mathsf{root}})$. Consider a non-empty set of vertices $S \subseteq \mathsf{V}(T)$, and let $i_{\min}$ and $i_{\max}$ denote the lowest and highest indices in L corresponding to vertices in S, respectively: $i_{\min} := \min \bigcup_{v \in S} L(v)$ and $i_{\max} := \max \bigcup_{v \in S} L(v)$.*

Then, for any i satisfying $i_{\min} \leq i \leq i_{\max}$, the path $P(v_{\mathsf{root}}, L_i)$ intersects the convex hull of S; that is, $\mathsf{V}(P(v_{\mathsf{root}}, L_i)) \cap \langle S \rangle \neq \varnothing$.

Proof. Let lca be the lowest common ancestor of $L_{i_{\min}}$ and $L_{i_{\max}}$. By definition $L_{i_{\min}}, L_{i_{\max}} \in S$, and, as lca lies on the unique path connecting these two vertices in T, we have $\mathsf{lca} \in \langle S \rangle$. In the following, we show that, for each i satisfying $i_{\min} \leq i \leq i_{\max}$, lca is one of the vertices in path $P(v_{\mathsf{root}}, L_i)$, which implies that $P(v_{\mathsf{root}}, L_i)$ intersects the convex hull of S.

If $a := \min L(\mathsf{lca})$ and $b := \max L(\mathsf{lca})$, Lemma 3 guarantees that a vertex v is in the subtree rooted at lca if and only if $L(v) \subseteq [a, b]$. Then, since $L_{i_{\min}}$ and $L_{i_{\max}}$ are in lca's subtree, we know that $i_{\min}, i_{\max} \in [a, b]$.

Moreover, by hypothesis $i_{\min} \leq i \leq i_{\max}$, so $i \in [a, b]$ as well. By the property of L just stated (from Lemma 3), it follows that L_i *lies in the subtree rooted at* lca. Equivalently, lca is an ancestor of L_i in T. Finally, since lca is an ancestor of L_i, the unique path from v_{root} to L_i *must* pass through lca. Therefore $\mathsf{V}(P(v_{\mathsf{root}}, L_i)) \cap \langle S \rangle \neq \varnothing$, which concludes our proof. □

Subprotocol $\mathsf{PathsFinder}^0$. We now present our subprotocol $\mathsf{PathsFinder}^0$. Each party computes the list representation L of the (rooted) input space tree T using the subprotocol $\mathsf{ListConstruction}$, which provides all parties with the same list L.

Afterwards, the parties execute $\mathsf{RealAA}(1)$ on the list L: each party joins $\mathsf{RealAA}(1)$ with an index in L that corresponds to its input vertex v_{IN}. $\mathsf{RealAA}(1)$ then provides the parties with indices corresponding to 1-close vertices v in T. Finally, each honest party returns the path from the root vertex v_{root} to the vertex obtained.

$\mathsf{PathsFinder}^0(T, v_{\mathsf{root}}, v_{\text{IN}})$

Code for party p, given the labeled tree T with root vertex v_{root} and input $v_{\text{IN}} \in \mathsf{V}(T)$

1: $L := \mathsf{ListConstruction}(T, v_{\mathsf{root}})$.
2: Join $\mathsf{RealAA}(1)$ with input $i := \min L(v_{\text{IN}})$; obtain output $j \in \mathbb{R}$.
3: **return** $P := P(v_{\mathsf{root}}, L_{[j]})$.

We establish the guarantees of $\mathsf{PathsFinder}^0$ below.

Lemma 5. *Assume a protocol RealAA achieving $\mathcal{AA}$ on $\mathbb{R}$ with round complexity $R_{\mathsf{RealAA}}(D, \varepsilon)$ when the honest inputs are D-close and the target error is ε. Then, if the honest parties join $\mathsf{PathsFinder}^0$ with the same labeled tree T and the same root vertex v_{root}, $\mathsf{PathsFinder}^0$ provides each honest party with a subpath P of T such that:*

1. *If S denotes the set of honest parties' inputs v_{IN}, then $\mathsf{V}(P) \cap \langle S \rangle \neq \varnothing$.*
2. *There exists a subpath $P^\star = (v_1 = v_{\mathsf{root}}, v_2, \ldots, v_{k^\star+1})$ of T such that, for every honest party, either $P = (v_1, v_2, \ldots, v_{k^\star})$ or $P = (v_1, v_2, \ldots, v_{k^\star+1})$.*

Moreover, each honest party obtains its path within $R_{\mathsf{PathsFinder}^0} := R_{\mathsf{RealAA}}(2 \cdot \left|\mathsf{V}(T)\right|, 1)$ rounds.

Proof. We first discuss Property 1. Lemma 3 ensures that every vertex of T appears in list L, and therefore every party obtains a well-defined index i. Afterwards, $\mathsf{RealAA}(1)$ ensures that parties obtain 1-close real values j that lie within the range of honest indices i. According to Remark 1, the values $\lfloor j \rfloor$ are also within the range of honest indices i. Then, since the honest parties have computed the list L identically, Lemma 4 ensures that each honest party obtains a path P that intersects the honest inputs' convex hull; therefore, Property 1 holds.

For Property 2, we note that, since the real values j obtained by the honest parties are 1-close, Remark 1 ensures that the integer values $\lfloor j \rfloor$ are also 1-close. Moreover, Lemma 3 ensures that consecutive elements in list L are adjacent vertices in T. Then, since the parties have computed the list representation L identically, the parties obtain vertices $L_{\lfloor j \rfloor}$ that are 1-close in T.

If the honest parties have obtained the same vertex $L_{\lfloor j \rfloor}$, then all honest parties obtain $P^\star = P(v_{\mathsf{root}}, L_{\lfloor j \rfloor})$, and hence Property 2 holds. Otherwise, honest parties have obtained adjacent vertices v, v'. Assume, without loss of generality, that v is the parent of v' in the tree T rooted at v_{root}. We may then define $P^\star$ as the "longer" path obtained by the honest parties, i.e., $P^\star := P(v_{\mathsf{root}}, v')$: parties that have obtained vertex v' return path $P^\star$, while parties that have obtained vertex v return path $P(v_{\mathsf{root}}, v)$ such that $v' \notin \mathsf{V}(P(v_{\mathsf{root}}, v))$ (since v is the parent of v') and $P^\star = P(v_{\mathsf{root}}, v) \oplus (v, v')$. Consequently, Property 2 holds in this case as well.

Finally, the number of rounds follows from the round complexity of RealAA, using the fact that we run RealAA with $\varepsilon := 1$ on inputs between 1 and $|L|$, and $|L| \leq 2 \cdot |\mathsf{V}(T)|$ according to Lemma 3. □

4.3.2 Protocol

We now combine the components described in Section 4.1 and Section 4.3.1 to present an almost-optimal protocol achieving $\mathcal{AA}$ on trees, denoted by TreeAA^0.

The parties begin by fixing the root vertex v_{root} of the input space tree T as the vertex with the lowest label in lexicographic order. Afterwards, they run the subprotocol $\mathsf{PathsFinder}^0$ described in Section 4.3.1 to *approximately* agree on paths that intersect the honest inputs' convex hull, as described by Lemma 5.

Once these paths are obtained, each party p proceeds according to the approach described in Section 4.2: it denotes the k vertices on its own path P as $(v_1 := v_{\mathsf{root}}, v_2, \ldots, v_k := v)$. It joins $\mathsf{RealAA}(1)$ with input i, where the vertex denoted by v_i in p's path P is the projection of p's input vertex v_{IN} onto P; formally $v_i = \mathsf{proj}_P(v_{\text{IN}})$.

Upon obtaining an output j from $\mathsf{RealAA}(1)$, party p *should* output the vertex denoted by $v_{\lfloor j \rfloor}$ in its path P. However, this is the point where we need to be careful about honest parties holding different paths: an honest party p might obtain $j > k$ from $\mathsf{RealAA}(1)$. If this is the case, then p holds the "shorter" path $(v_1, v_2, \ldots, v_k)$, while other honest parties hold the "longer" path $P^\star = (v_1, v_2, \ldots, v_{k+1})$, as described in Lemma 5. In this case, if $\lfloor j \rfloor = k + 1$ and vertex v_k has at least three neighbors (as illustrated in Figure 4) party p cannot uniquely determine which of the neighbors corresponds to v_{k+1} – the last vertex of the "longer path".

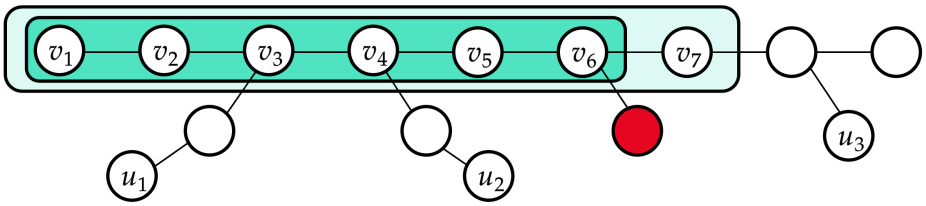


Figure 4: In this figure, vertices u_1, u_2, u_3 are the honest inputs, and the highlighted paths $(v_1, v_2, \ldots, v_6)$ and $(v_1, v_2, \ldots, v_7)$ represent the paths P that the honest parties obtained via $\mathsf{PathsFinder}^0$. Note that an honest party p that holds $P = (v_1, \ldots, v_6)$ might obtain a value j such that $\lfloor j \rfloor = 7$, and p does not know whether v_7 should be the actual vertex v_7 or the red vertex adjacent to v_6. The red vertex is, in fact, outside the honest inputs' convex hull.

In TreeAA^0, if party p obtains $j > k$, it simply outputs v_k. We will show that, in this case, all honest parties output either v_k or, if they hold the longer path P, v_{k+1}, and therefore $\mathcal{AA}$ is achieved.

We present the code of TreeAA^0 below.

TreeAA^0

Code for party p with input $v_{\text{IN}} \in \mathsf{V}(T)$

1: $v_{\mathsf{root}} :=$ the vertex in T with the lowest label (in lexicographic order).
2: $P := \mathsf{PathsFinder}^0(T, v_{\mathsf{root}}, v_{\text{IN}})$.
3: Denote the $k = |V(P)|$ vertices in P by $(v_1 := v_{\mathsf{root}}, v_2, \ldots, v_k)$.
4: Wait until round $R_{\mathsf{PathsFinder}^0}$ ends.
5: Join $\mathsf{RealAA}(1)$ with input i such that $\mathsf{proj}_P(v_{\text{IN}}) = v_i$; obtain output j.
6: If $\lfloor j \rfloor > k$, output the vertex denoted by v_k in $\mathsf{V}(P)$; otherwise, output the vertex denoted by $v_{\lfloor j \rfloor}$ in $\mathsf{V}(P)$.

Note that in line 4, the parties wait for a specific number of rounds, namely $R_{\mathsf{PathsFinder}^0}$, before initiating the execution of RealAA in line 5. This is because $\mathsf{PathsFinder}^0$ – which uses RealAA as a building block – does not guarantee that all the honest parties obtain their paths in the same round (as briefly mentioned in Section 4.1). Instead, it only guarantees that all the honest parties obtain their

paths within $R_{\mathsf{PathsFinder}^0}$ rounds. Hence, by waiting until the end of round $R_{\mathsf{PathsFinder}^0}$, all the honest parties can start executing RealAA in line 5 simultaneously.

The theorem below describes the guarantees provided by protocol TreeAA^0.

Theorem 3. *Assume a protocol RealAA achieving $\mathcal{AA}$ on $\mathbb{R}$ with round complexity $R_{\mathsf{RealAA}}(D, \varepsilon)$ when the honest inputs are D-close and the target error is ε.*

Even when up to $t < n$ of the n parties involved are Byzantine, protocol TreeAA^0 achieves $\mathcal{AA}$ on any input space tree T within $R_{\mathsf{RealAA}}(2 \cdot \left|V(T)\right|, 1) + R_{\mathsf{RealAA}}(D(T), 1)$ rounds.

Proof. We first note that Lemma 5 ensures that the honest parties obtain their paths P within $R_{\mathsf{PathsFinder}^0}$ rounds, and therefore the honest parties start the execution of $\mathsf{RealAA}(1)$ simultaneously in line 5 of the protocol. Then $\mathsf{RealAA}(1)$ guarantees that the honest parties obtain 1-close real values j that are within the range of honest inputs i. Using Remark 1, we may establish that the same properties hold for the integer values $\lfloor j \rfloor$ obtained by the honest parties; these are also in the range of honest inputs i, and are also 1-close.

As the honest values i are $\mathsf{D}(T)$-close, the honest parties obtain values j within $R_{\mathsf{RealAA}}(\mathsf{D}(T), 1)$ rounds. This implies that TreeAA^0 achieves Termination within $R_{\mathsf{PathsFinder}^0} + R_{\mathsf{RealAA}}(\mathsf{D}(T), 1)$ rounds. Afterwards, using Lemma 5, we obtain that TreeAA^0 achieves Termination within $R_{\mathsf{RealAA}}(\left|\mathsf{V}(T)\right|, 1) + R_{\mathsf{RealAA}}(\mathsf{D}(T), 1)$ rounds.

In order to discuss Validity and 1-Agreement, it is important to establish that, in line 3, the honest parties denote the vertices of their paths P in a consistent manner. According to Lemma 5, and, as depicted in Figure 4, there are two 1-close vertices v and v' with the following properties:

(i) v is the parent of v' (with respect to the root vertex v_{root});

(ii) every honest party obtains a path P such that either $P = P(v_{\mathsf{root}}, v)$ or, if $v \neq v'$, $P = P(v_{\mathsf{root}}, v') = P(v_{\mathsf{root}}, v) \oplus (v, v')$.

Then all honest parties denote the vertices in $P(v_{\mathsf{root}}, v)$ identically. In addition, honest parties holding path $P(v_{\mathsf{root}}, v')$ denote vertex v' identically.

We may now discuss Validity. Lemma 5 ensures that the paths P the honest parties obtain (hence, both $P(v_{\mathsf{root}}, v)$ and $P(v_{\mathsf{root}}, v')$) intersect the honest inputs' convex hull. Applying Lemma 2, this implies that, for every honest party, $\mathsf{proj}_P(v_{\text{IN}})$ is a valid vertex. Hence, for every honest party's index i, the vertex denoted by v_i in $P(v_{\mathsf{root}}, v')$ is a valid vertex. Moreover, if $i_{\min}$ and $i_{\max}$ represent the lowest and the highest honest indices i, then all vertices denoted by $v_{i_{\min}}, \ldots, v_{i_{\max}}$ in $P(v_{\mathsf{root}}, v')$ are valid. We recall that honest parties obtain integer values $\lfloor j \rfloor \in [i_{\min}, i_{\max}]$. If the condition $\lfloor j \rfloor \leq k$ holds for an honest party p (i.e., it knows which vertex is denoted by $v_{\lfloor j \rfloor}$), then it follows immediately that p outputs a valid vertex $v_{\lfloor j \rfloor}$. Otherwise, if an honest party p has obtained $\lfloor j \rfloor > k$, then p holds the *shorter* path P: $v \neq v'$, p holds the path $P(v_{\mathsf{root}}, v)$ of $k^\star$ vertices. Moreover, there is an honest party p' that has obtained the path $P(v_{\mathsf{root}}, v) \oplus (v, v')$ of $k^\star + 1$ vertices, and has joined $\mathsf{RealAA}(1)$ with index $i = k^\star + 1$. This implies that $k^\star \in [i_{\min}, i_{\max}]$ and therefore p's vertex v_k (where $k = k^\star$) is a valid vertex. Consequently, TreeAA^0 achieves Validity.

For 1-Agreement, we recall that honest parties obtain 1-close integer values $\lfloor j \rfloor$. Therefore, the vertices denoted by $v_{\lfloor j \rfloor}$ in $P(v_{\mathsf{root}}, v')$ are 1-close. This implies that, if the condition $\lfloor j \rfloor \leq k$ is satisfied for all honest parties, 1-Agreement is achieved. We still need to discuss the case where some honest party p obtains $\lfloor j \rfloor > k$: again, this means that $v \neq v'$, p holds the path $P(v_{\mathsf{root}}, v)$ of $k^\star$ vertices, and there is an honest party p' that holds the path $P(v_{\mathsf{root}}, v) \oplus (v, v')$ of $k^\star + 1$ vertices. In this case, we show that all honest parties obtain $\lfloor j \rfloor \in \{k^\star, k^\star + 1\}$. First, since the honest parties' integer values $\lfloor j \rfloor$ are 1-close, every honest party has obtained $\lfloor j \rfloor \geq k^\star$. Second, Lemma 5 ensures that no honest party holds a path P consisting of more than $k^\star + 1$ vertices, which implies that $i_{\max} \leq k^\star + 1$. As the honest parties' integer values $\lfloor j \rfloor$ are in $[i_{\min}, i_{\max}]$, every honest party obtains $\lfloor j \rfloor \leq k^\star + 1$. Therefore, every honest party obtains $\lfloor j \rfloor \in \{k^\star, k^\star + 1\}$. Hence, as every honest party holds either $k = k^\star$ or $k = k^\star + 1$, every honest party

outputs either $v_{k^\star}$ or $v_{k^\star+1}$, so 1-Agreement holds. We may therefore conclude that TreeAA^0 achieves $\mathcal{AA}$. □

By instantiating RealAA with the protocol described in Theorem 2, we obtain the following:

Corollary 1. *There is a protocol TreeAA^0 achieving $\mathcal{AA}$ on any input space tree T whenever up to $t < n/3$ of the n parties involved are Byzantine, within $O\left(\frac{\log|\mathsf{V}(T)|}{\log\log|\mathsf{V}(T)|}\right)$ rounds.*

Considering our lower bound presented in Section 3, this solution is not yet optimal: given that $t \in \Theta(n)$, it matches the lower bound whenever $\log \mathsf{D}(T) \in \Theta(\log \mathsf{V}(T))$, hence for trees T with diameter $\mathsf{D}(T) \in |\mathsf{V}(T)|^{\Theta(1)}$. Combining this with the protocol of [37] that solves $\mathcal{AA}$ on trees within $O(\log \mathsf{D}(T))$ rounds, we have optimal protocols whenever the input space tree's diameter is either constant or polynomial in the number of vertices (for some constant exponent ≤ 1). Thus, this leaves a gap for trees T of diameter $\mathsf{D}(T) \in \left[\omega(1), |\mathsf{V}(T)|^{o(1)}\right]$. In the subsequent section, we give an improved protocol that closes this gap.

4.4 Weakening the Path Assumption: Optimal Solution

We observe that the idea of Section 4.2 does not require the honest parties to *know the same path* that intersects the honest inputs' convex hull. As pointed out in Section 4.3, what the argument needs is only a weak form of pre-agreement. We now explore an even weaker form of pre-agreement, which leads to our final solution for $\mathcal{AA}$ on trees. Concretely, each honest party should hold two paths P and Q, which may be distinct over the parties. However, the following should hold: (i) every honest party's path P intersects the honest inputs' convex hull; (ii) every honest party's path Q has every honest party's path P as a prefix, as defined below, and (iii) every honest party can map the value j obtained via RealAA to a valid vertex $v_{\lfloor j \rfloor}$. Figure 5 depicts these assumptions.

Definition 3 (Path Prefix)**.** *A path $P = (v_0, \ldots, v_k)$ is a* prefix *of a path $Q = (u_0, \ldots, u_\ell)$ if $k \leq \ell$ and $v_i = u_i$ for all $i \in \{0, \ldots, k\}$.*

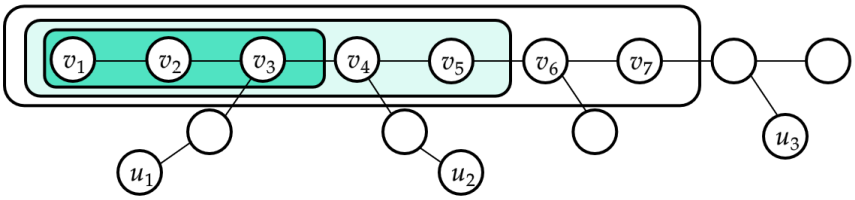


Figure 5: An honest party p may hold a path Q such as $(v_1, \ldots, v_7)$. Moreover, all honest parties' paths P are guaranteed to be prefixes of p's path Q, such as those highlighted in cyan and light cyan: (v_1, v_2, v_3), and $(v_1, v_2, \ldots, v_5)$. Note that the honest parties' paths P intersect the honest inputs' convex hull $\langle\{u_1, u_2, u_3\}\rangle$.

We explain how we can provide each honest party with such paths P and Q in Section 4.5. Given these paths, we proceed as follows: each honest party denotes the ℓ vertices of its path Q by $(v_1, \ldots, v_\ell)$. It joins $\mathsf{RealAA}(1)$ with input i, where v_i denotes the projection of its input vertex onto the path P, i.e., $\mathsf{proj}_P(v_{\text{IN}})$. Protocol $\mathsf{RealAA}(1)$ returns real values j that are 1-close and lie within the range of the honest inputs i, so by Remark 1 the rounded integers $\lfloor j \rfloor$ are also 1-close and within that same range. Finally, each party p outputs the vertex $v_{\lfloor j \rfloor}$ on its path Q: note that $\lfloor j \rfloor$ is at most the length of the longest honest path P, and therefore p has assigned a label for it.

TreeAAFromPaths(T)

Code for party p with input $v_{\text{IN}} \in \mathsf{V}(T)$, and with paths P, Q in T (as defined in Lemma 6)

1: Denote the $\ell = \left|\mathsf{V}(Q)\right|$ vertices in the (directed) path Q by $(v_1, v_2, \ldots, v_\ell)$.
2: Join RealAA(1) with input i such that $\mathsf{proj}_P(v_{\text{IN}}) = v_i$; obtain output j.
3: Output the vertex denoted by $v_{[j]}$ in $\mathsf{V}(Q)$.

We will prove that TreeAAFromPaths(T) achieves $\mathcal{AA}$ given that all honest parties hold such paths P and Q. In Section 4.5 we explain how to obtain such paths, and we put everything together in Section 4.6.

Lemma 6. *Let T be a labeled tree of diameter D. Suppose each party is provided with two paths P and Q (that may be distinct across the parties) in T such that:*

1. *every honest party's path P intersects the honest inputs' convex hull;*
2. *every honest party's path P is a prefix of its path Q, and if two honest parties hold (P, Q) and (P', Q') respectively, then P is a prefix of Q'.*

Then, for any protocol RealAA that achieves $\mathcal{AA}$ on $\mathbb{R}$ in $R_{\mathsf{RealAA}}(D, \varepsilon)$ rounds when the honest inputs are D-close and the target error is ε, there is a protocol that solves $\mathcal{AA}$ on T in $R_{\mathsf{RealAA}}(D, 1)$ rounds.

We split the proof of Lemma 6 into multiple steps. Each of the lemmas below makes the same assumptions on the honest parties' paths P and Q as Lemma 6. We first note that these assumptions immediately imply that the parties assign consistent labels to the vertices in their paths Q, as stated below.

Remark 2. *Let $P^\star$ denote the longest path P that an honest party joins TreeAAFromPaths(T) with, and denote its vertices by $(v_1^\star, \ldots, v_{k_{\max}}^\star)$. Then, for every honest party p that has labeled the vertices in its path Q by $(v_1, \ldots, v_\ell)$, the following hold: $\ell \geq k_{\max}$ and $v_i = v_i^\star$ for every i with $1 \leq i \leq k_{\max}$.*

Next, we need to establish that the honest parties join RealAA with well-defined values i.

Lemma 7. *Let $k_{\max}$ denote the number of vertices in the longest path P that an honest party joins TreeAAFromPaths(T) with. Then, every honest party obtains an integer value i satisfying $1 \leq i \leq k_{\max}$.*

Proof. The assumptions of Lemma 6 ensure that honest parties' paths P are non-empty, hence $\mathsf{proj}_P(v_{\text{IN}})$ is well-defined. Then, since P contains $k \leq k_{\max}$ vertices denoted by $(v_1, v_2, \ldots, v_k)$, every honest party obtains a vertex $v_i = \mathsf{proj}_P(v_{\text{IN}})$ such that i is an integer satisfying $1 \leq i \leq k_{\max}$. □

In order to show that TreeAAFromPaths(T) achieves 1-Agreement and Validity, we discuss the properties for the integer values $[j]$ obtained by the honest parties in Lemma 8. Afterwards, Lemma 9 translates these guarantees back to the original input space tree T.

Lemma 8. *In line 2, the honest parties obtain values j such that the resulting integer values $[j]$ are 1-close and within the range of their values i.*

Proof. By Lemma 7, the honest parties obtain well-defined real values i, and then they simultaneously invoke RealAA(1). Now, Theorem 2 applied with $\varepsilon = 1$ guarantees that the honest parties obtain 1-close real values j that lie within the range of the honest values i. Finally, Remark 1 implies that (1) the rounded values $[j]$ are still pairwise within distance 1, and (2) each $[j]$ lies in the honest range of values i, as claimed. □

Lemma 9. *The honest parties' outputs are 1-close vertices in the convex hull of the honest inputs.*

Proof. To prove the statement, consider an arbitrary honest party p and map its output $v_{\lfloor j \rfloor_p}$ onto the path $P^\star$ described in Remark 2. Recall that $P^\star$ is the longest path P that some honest party has joined $\mathsf{TreeAAFromPaths}(T)$ with. We denote the vertices of $P^\star$ by $(v_1^\star, \ldots, v_{k_{\max}}^\star)$.

By Lemma 7, each honest party p obtains a value i_p satisfying $1 \leq i_p \leq k_{\max}$. Moreover, Lemma 8 ensures that for any two honest parties p and p', the integer values $\lfloor j \rfloor_p$ and $\lfloor j \rfloor_{p'}$ are 1-close. Let $i_{\min} := \min_h i_h$ and $i_{\max} := \max_h i_h$ be the minimum and maximum i_h value over all honest parties h. For every honest p, we have that $i_{\min} \leq \lfloor j \rfloor_p \leq i_{\max} \leq k_{\max}$. Furthermore, Remark 2 implies that for every honest party p and every $k \leq k_{\max}$ we have $v_k = v_k^\star$. Hence $v_{\lfloor j \rfloor_p} = v^\star_{\lfloor j \rfloor_p}$, and therefore, the honest parties' outputs are pairwise 1-close along $P^\star$.

For Validity, by the assumptions of Lemma 6, each honest party's path P_p intersects convex hull of the honest inputs. By Lemma 2, every projection vertex v_{i_p} lies in the convex hull of honest inputs. Consequently, for every index k satisfying $i_{\min} \leq k \leq i_{\max}$, the vertex $v_k^\star$ is in the convex hull of the honest inputs. Since $\lfloor j \rfloor_p$ lies within the range of honest values, we obtain that each honest party p outputs the vertex $v_{\lfloor j \rfloor_p} = v^\star_{\lfloor j \rfloor_p}$ in the convex hull of the honest inputs. □

We conclude the subsection by presenting the proof of Lemma 6.

Proof of Lemma 6. We first discuss Termination. Given that each party is provided with two paths P and Q with the properties described in the lemma's statement, Lemma 7 guarantees each honest party obtains a well-defined value i with $1 \leq i \leq k_{\max} \leq \mathsf{D}$. The parties then invoke $\mathsf{RealAA}(1)$ (simultaneously) on those values, and hence the total round complexity is $R_{\mathsf{RealAA}}(\mathsf{D}, 1)$. The 1-Agreement and Validity properties follow directly from Lemma 9. Therefore, $\mathsf{TreeAAFromPaths}(T)$ satisfies all requirements of $\mathcal{AA}$. □

4.5 Finding These Paths

We now present our protocol $\mathsf{PathsFinder}(T)$, providing the parties with paths satisfying the preconditions of $\mathsf{TreeAAFromPaths}$ (described in Lemma 6) within $O(1)$ rounds.

Each party joins $\mathsf{PathsFinder}(T)$ with its input vertex v_{IN}. Then, we fix some vertex v_{start} (e.g., the vertex in T with the lowest label in lexicographic order) and each party communicates the path from v_{start} to its input vertex $P(v_{\mathsf{start}}, v_{\text{IN}})$. Each party will try to define its path P roughly as the *longest common prefix* of the paths $P(v_{\mathsf{start}}, v_{\text{IN}})$ it has received: note that the honest parties might not have an identical view over the paths $P(v_{\mathsf{start}}, v_{\text{IN}})$ due to the Byzantine parties. Then, each party will define its path Q roughly as what it *believes* other honest parties might have identified as a longest common prefix. We formalize this idea with the help of a mechanism called Gradecast ($\mathcal{GC}$), which ensures that parties distribute their paths in a consistent manner, i.e., Byzantine parties are unable to send different messages to different parties, but they can still cause inconsistencies by making sure that only a subset of the honest parties receive their messages. We note that $\mathcal{GC}$ delivers each message with a grade in $\{0, 1, 2\}$. Honest parties' messages are always delivered with grade 2. In addition, if two parties receive messages with grades g and g' from a party p, then $\left|g - g'\right| \leq 1$. Moreover, if both g and g' are greater than zero, then the two parties have received the same message from p. We state the formal definition of $\mathcal{GC}$ below, as presented in [7].

Definition 4. *(Gradecast) Let Π be a protocol where a designated party S (the sender) holds a value v_S. We say that Π achieves $\mathcal{GC}$ if the following properties hold even when t out of the n parties involved are Byzantine:* ***(Termination)*** *Every honest party eventually outputs a pair (v, g), where v is a message and $g \in \{0, 1, 2\}$;* ***(Integrity)*** *If S is honest, then every honest party outputs $(v_S, 2)$;* ***(Consistency)*** *If two honest parties output (v, g) and (v', g') then $\left|g - g'\right| \leq 1$. In addition, if $g, g' > 0$, then $v = v'$.*

We then make use of the $\mathcal{GC}$ protocol of [7] (presented in the full version [8]), described below.

Theorem 4 (Theorem 1 of [8]). *There is a protocol GC achieving $\mathcal{GC}$ even when up to $t < n/3$ of the n parties involved are Byzantine, with round complexity $R_{\mathsf{GC}} := 3$.*

Hence, each party will distribute its path $P(v_{\mathsf{start}}, v_{\text{IN}})$ via GC. Every party obtains at least $n-t$ paths with grade 2 (from honest parties), plus up to t paths with various grades. Then, each party defines its path P as a path prefix *supported* by $n-t$ parties with grade 2, as defined below.

Definition 5 (Supported Path Prefix). *Let $\{P_1, P_2, \ldots, P_k\}$ be the collection of paths. We say that a path P is* supported by ℓ parties *if $\big|\{i : 1 \leq i \leq k, P \text{ is a prefix of } P_i\}\big| \geq \ell$. The* path prefix supported by ℓ parties *is defined to be the longest path P satisfying the above property.*

The paths Q are defined as path prefixes supported by $n-t$ parties with grade at least 1 instead of 2, i.e., some honest parties may have received these supports with grade 2.

$\mathsf{PathsFinder}(T)$

Code for party p with input $v_{\text{IN}} \in \mathsf{V}(T)$

1: Send $P(v_{\mathsf{start}}, v_{\text{IN}})$ to all parties via GC, where v_{start} is the vertex in T with the lexicographically smallest label.
2: Let $P :=$ the longest path prefix supported by at least $n-t$ parties with grade 2.
3: Let $Q :=$ the longest path prefix supported by at least $n-t$ parties with grade at least 1.
4: Output (P, Q).

The lemma below presents the guarantees of $\mathsf{PathsFinder}(T)$.

Lemma 10. *Assume a protocol GC that achieves $\mathcal{GC}$ within R_{GC} rounds. Then, if $t < n/2$, $\mathsf{PathsFinder}(T)$ provides each honest party within R_{GC} rounds with two non-empty paths P and Q such that: (1) P intersects the honest inputs' convex hull; (2) P is a prefix of Q. Moreover, if two honest parties obtain (P, Q) and (P', Q') respectively, then P is a prefix of Q'.*

Proof. Let LCP be the longest common prefix of the paths $P(v_{\mathsf{start}}, v_{\text{IN}})$ held by the honest parties. Lemma 11, stated below, ensures that LCP is non-empty and ends at a valid vertex v_{LCP}.

For the first property, we show that every honest party's path P has LCP as a prefix. We recall that every honest party p defines P as the longest path prefix supported by at least $n-t$ paths received with grade 2. Since the honest parties' paths have prefix LCP, p receives at least $n-t$ paths with prefix LCP with grade 2 due to the definition of $\mathcal{GC}$ (Definition 4). In addition, the honest parties receive up to $t < n-t$ (since $t < n/2$) paths that do not have LCP as a prefix. Hence, the longest path prefix supported by at least $n-t$ parties with grade 2 in p's view must have LCP as a prefix, and therefore P has prefix LCP. As LCP ends at a valid vertex v_{LCP}, we may conclude that P intersects the honest inputs' convex hull.

For the second property, assume that an honest party p has obtained a path P in line 2, and an honest party p' has obtained a path Q' in line 3. We prove that P is a prefix of Q': we show that p' receives support from at least $n-t$ parties for the path prefix P, and does not receive sufficient support for any other path prefix. We recall once again that the path P was chosen by p as the longest prefix supported by at least $n-t$ paths received with grade 2. Following the definition of $\mathcal{GC}$, we know that if p receives a message with grade 2, all honest parties get the same message with grade at least 1. This guarantees that p' has received at least $n-t$ paths with grade at least 1 that have P as a prefix, and up to $t < n-t$ (since $t < n/2$) messages supporting any path prefix different from P. Hence, p' defines Q' as a path with prefix P. Note that this holds if $p = p'$ as well.

We still need to discuss the round complexity: $\mathsf{PathsFinder}(T)$ makes n parallel invocations of GC, and therefore terminates within R_{GC} rounds. □

Lemma 11. *Let $S \subseteq \mathsf{V}(T)$ and consider two vertices $v_{\mathsf{end}}, v'_{\mathsf{end}}$ in $\langle S \rangle$. Then, for any $v_{\mathsf{start}} \in \mathsf{V}(T)$, the longest common prefix of $P(v_{\mathsf{start}}, v_{\mathsf{end}})$ and $P(v_{\mathsf{start}}, v'_{\mathsf{end}})$ is non-empty and ends in $\langle S \rangle$.*

Proof. Let $P(v_{\mathsf{start}}, v)$ be the longest common prefix of $P(v_{\mathsf{start}}, v_{\mathsf{end}})$ and $P(v_{\mathsf{start}}, v'_{\mathsf{end}})$. This is non-empty, as both paths start with vertex v_{start}. Note that, by rooting the tree T at v_{start}, vertex v becomes the lowest common ancestor of v_{end} and v'_{end} and therefore lies on the unique path $P(v_{\mathsf{end}}, v'_{\mathsf{end}})$. Then, as $v_{\mathsf{end}}, v'_{\mathsf{end}} \in \langle S \rangle$, we obtain that $v \in \langle S \rangle$. □

4.6 Putting it all together

In the following, we present our final protocol for $\mathcal{AA}$ on trees. The parties run PathsFinder, which provides them with paths meeting the preconditions of TreeAAFromPaths. Afterwards, the parties run TreeAAFromPaths, which provides them with the final outputs.

TreeAA(T)

Code for party p with input $v_{\text{IN}} \in \mathsf{V}(T)$

1: Join PathsFinder(T) with input v_{IN} and obtain paths P and Q.
2: Run TreeAAFromPaths(T) with inputs v_{IN}, P, Q to compute v_{OUT}; then output v_{OUT} and terminate.

The theorem below follows directly from Lemma 10 and Lemma 6.

Theorem 5. *Assume a $\mathcal{GC}$ protocol GC with round complexity R_{GC}, and a protocol RealAA achieving $\mathcal{AA}$ on $\mathbb{R}$ with round complexity $R_{RealAA}(D, \varepsilon)$ when the honest inputs are D-close and the target error is ε. Then, if $t < n/2$, TreeAA(T) achieves $\mathcal{AA}$ on any input space tree T within $R_{GC} + R_{RealAA}(\mathsf{D}(T), 1)$ rounds.*

We obtain our final corollary by instantiating GC with the protocol described in Theorem 4 and RealAA with the protocol described in Theorem 2.

Corollary 2. *There is a synchronous protocol that achieves $\mathcal{AA}$ on any input space tree T within $O\left(\frac{\log \mathsf{D}(T)}{\log \log \mathsf{D}(T)}\right)$ rounds even when up to $t < n/3$ of the n parties involved are Byzantine.*

A note on the $t < n/2$ case. We have presented our protocol TreeAA(T) under the assumption that $t < n/3$: this is the optimal resilience threshold in an *unauthenticated setting* (i.e., no cryptographic assumptions) [15]. This setting was chosen solely for simplicity of presentation. In fact, our reduction can be adapted to honest-majority *authenticated* settings as well. The term *authenticated* refers to settings assuming a public key infrastructure (PKI) and a secure digital signature scheme. Concretely, we may instantiate GC with the protocol of [35], described in Theorem 6.

Theorem 6 (Theorem 4.1 of [35])**.** *There is a protocol GC^{PKI} achieving $\mathcal{GC}$ in an authenticated setting even when up to $t < n/2$ of the n parties involved are Byzantine, with round complexity $R_{GC^{PKI}} := 3$.*

For RealAA, we obtain the result below by combining two protocols: the $\mathcal{AA}$ protocol of [27] and the Proxcensus protocol of [23]. After making minor adjustments, the protocol of [23] becomes an $\mathcal{AA}$ protocol with round complexity $O\left(\frac{\log(\mathsf{D}/\varepsilon)}{\log\log(\mathsf{D}/\varepsilon)}\right)$ whenever $\mathsf{D}/\varepsilon = \omega(1)$. If $\mathsf{D}/\varepsilon = O(1)$ instead, we make use of the protocol of [25], which achieves Termination within $O(\log(\mathsf{D}/\varepsilon)) = O(1)$ rounds. We present this formally in Section C.

Theorem 7. *If the honest inputs are D-close real values, there is a protocol $RealAA^{PKI}(D, \varepsilon)$ achieving $\mathcal{AA}$ in authenticated settings even when up to $t = c \cdot n$ (where c is a constant in $[1/3, 1/2)$) of the n parties are Byzantine. $RealAA^{PKI}(D, \varepsilon)$ achieves Termination within $R_{RealAA^{PKI}}(D, \varepsilon) = O\left(\frac{\log(D/\varepsilon)}{\log\log(D/\varepsilon)}\right)$ rounds.*

This yields a protocol with optimal round complexity up to $t = c \cdot n$ corruptions for any constant $c \in [1/3, 1/2)$, assuming a public key infrastructure and digital signatures, i.e., in the authenticated setting. This leads to the following corollary.

Corollary 3. *There is a synchronous protocol that solves $\mathcal{AA}$ on any input space tree T in $O\left(\frac{\log \mathsf{D}(T)}{\log\log \mathsf{D}(T)}\right)$ rounds in authenticated settings even when up to $t = c \cdot n$ of the n parties are Byzantine, where c is a constant in $[1/3, 1/2)$.*

We add that the cryptographic assumptions lead to a protocol secure against a *computationally bounded* adversary. When the digital signatures are replaced with real-world instantiations, the security of the protocol holds except with negligible probability (in the digital signature scheme's security parameter).

5 Extension to Block Graphs

In this section, we extend our algorithm from trees to connected block graphs. Our approach utilizes the clique tree representation to reduce the problem on a block graph to a problem on a tree, enabling us to achieve optimal round complexity for block graphs G. We first give the definition of block graphs and their clique tree structure, and afterwards present the reduction, and the resulting corollaries in the synchronous and asynchronous models.

Block Graphs. A graph $G = (V, E)$ is a *block graph* if every maximal 2-connected subgraph (block) of G is a clique. Equivalently, G is a chordal graph in which every two maximal cliques intersect in at most one vertex.

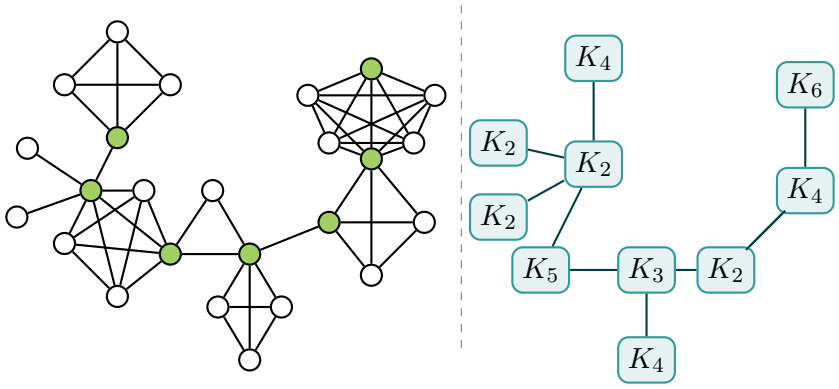


Figure 6: A block graph and its clique tree representation.

Clique trees. For any chordal graph G, hence also for any block graph G, its maximal cliques can be arranged as the vertices of a tree T, called the *clique tree* [10], as shown in Figure 6. We use $\mathsf{V}_G(c)$ to denote the set of vertices in the clique vertex $c \in \mathsf{V}(T)$. If clique vertices c, c' are adjacent in T, then $\mathsf{V}_G(c) \cap \mathsf{V}_G(c') \neq \varnothing$. The edges of T represent a subset of the non-empty intersection relation between maximal cliques in G, selected such that the *Running Intersection Property* holds: for any two maximal cliques c, c', the intersection $\mathsf{V}_G(c) \cap \mathsf{V}_G(c')$ is contained in every clique vertex on the unique path connecting them in T. Such a clique tree of minimum diameter can be computed deterministically and efficiently in $O(|V| + |E|)$ time for any chordal graph [11].

The following lemma states an outcome of the running intersection property on block graphs: every vertex in the intersection of at least two maximal cliques of G is a cut vertex in graph G.

Lemma 12 (Clique-Tree Edge Separator)**.** *Let G be a connected block graph and let T be a clique tree of G. Let $\{c, c'\}$ be an edge of T and let $u \in \mathsf{V}_G(c) \cap \mathsf{V}_G(c')$. Removing the edge $\{c, c'\}$ partitions T into two connected components T_c and $T_{c'}$ containing c and c', respectively. Let $V_c := \bigcup_{v \in \mathsf{V}(T_c)} \mathsf{V}_G(v)$ and $V_{c'} := \bigcup_{v \in \mathsf{V}(T_{c'})} \mathsf{V}_G(v)$. Then for any $x \in V_c \setminus \{u\}$ and any $y \in V_{c'} \setminus \{u\}$, every path in G from x to y passes through u. Consequently, u is a cut vertex of G.*

Proof. The proof of this lemma is given in the appendix (see Section D). □

Subdivided Clique Tree. Our protocol relies on a *subdivided clique tree* T_S of G. Given a clique tree T of G, the subdivided clique tree T_S is obtained by subdividing every edge $\{c, c'\}$ of T as follows: the edge $\{c, c'\}$ is replaced by two edges $\{c, c''\}$ and $\{c'', c'\}$, where c'' is a newly introduced intermediate vertex corresponding to the unique vertex in which the cliques c and c' intersect. By construction, the diameter of the subdivided clique tree satisfies $\mathsf{D}(T_S) = 2 \cdot \mathsf{D}(T)$. Note that this is similar to the *extended clique tree* defined by [37].

Protocol. We now present our $\mathcal{AA}$ protocol for connected block graphs. First, note that the labeled graph G is known to all parties. The parties then compute the same minimum-diameter labeled clique tree T for G using the algorithm guaranteed by [11], and hence the same minimum-diameter labeled subdivided clique tree T_S. The parties then run $\mathsf{TreeAA}(T_S)$ using as inputs the clique vertices in T_S containing their input vertices from G. If a vertex belongs to multiple cliques of G, the party selects arbitrarily one of the corresponding clique vertices in T_S. This provides each party with a vertex c_{OUT} in T_S which is either an intermediate vertex or a clique vertex. Each party then maps c_{OUT} to a vertex in the original graph G, which will be its output. We present the code below, and Theorem 8 presents the guarantees.

$\mathsf{BlockGraphAA}(G)$

Code for party p with input $v_{\text{IN}} \in \mathsf{V}(G)$

1: Let T_S be a minimum-diameter labeled subdivided clique tree of G.
2: Let c_{IN} be an arbitrarily chosen vertex in T_S such that $v_{\text{IN}} \in \mathsf{V}_G(c_{\text{IN}})$.
3: Join $\mathsf{TreeAA}(T_S)$ with input vertex c_{IN}. Obtain output vertex $c_{\text{OUT}} \in \mathsf{V}(T_S)$.
4: If c_{OUT} is a clique vertex, output $v_{\text{OUT}} :=$ the vertex $v \in \mathsf{V}_G(c_{\text{OUT}})$ with the shortest distance to v_{IN}.
5: Otherwise, c_{OUT} is an intermediate vertex. Let $c, c' \in \mathsf{V}(T_S)$ be the two vertices adjacent to c_{OUT} in T_S, and let v_{OUT} be the unique vertex in $\mathsf{V}_G(c) \cap \mathsf{V}_G(c')$. Output v_{OUT}.

Theorem 8. *Assume a protocol $\mathsf{TreeAA}(T)$ achieving $\mathcal{AA}$ on any tree T within $R_{\mathsf{TreeAA}}(\mathsf{D}(T))$ rounds. Then, $\mathsf{BlockGraphAA}(G)$ achieves $\mathcal{AA}$ on any connected block graph G within $R_{\mathsf{TreeAA}}(4 \cdot \mathsf{D}(G))$ rounds.*

We split the proof of Theorem 8 into multiple lemmas. The lemma below establishes Validity.

Lemma 13. *Every honest party p obtains a valid output vertex.*

Proof. Let $c_{\text{IN}} \in \mathsf{V}(T_S)$ be the clique vertex corresponding to p's input (so $v_{\text{IN}} \in V_G(c_{\text{IN}})$). By the Validity of $\mathsf{TreeAA}(T_S)$, the output c_{OUT} obtained by p lies in the convex hull of the honest inputs in T_S. That is, c_{OUT} lies on a path in T_S between two honest clique vertices c_a and c_b – these represent two maximal cliques in G. We split the proof into three cases, reflecting the output rule in the protocol. In the following, T denotes the clique tree of G from which T_S is constructed.

Case 1: c_{out} is an intermediate vertex, i.e., $c_{\text{out}} \in \mathbf{V}(T_S) \setminus \mathbf{V}(T)$. In this case, c_{OUT} subdivides an edge $\{c, c'\}$ of T. Party p outputs the unique vertex v_{OUT} in $\mathsf{V}_G(c) \cap \mathsf{V}_G(c')$. Since c_{OUT} lies on the unique path between c_a and c_b in T_S, the edge $e = \{c, c'\}$ must lie on the unique path between c_a and c_b in T. Removing the edge e separates T into two connected components, T_a (containing c_a) and T_b (containing c_b). By Lemma 12, this will imply that any path in G from an input vertex in c_a to an input vertex in c_b must pass through v_{OUT}. Thus, v_{OUT} lies on a shortest path between two honest inputs in G, and therefore lies in the convex hull of the honest inputs.

Case 2: c_{out} is a clique vertex, and $v_{\text{in}} \in \mathbf{V}_G(c_{\text{out}})$. Here, p outputs $v_{\text{OUT}} = v_{\text{IN}}$, which is trivially valid.

Case 3: c_{out} is a clique vertex, but $v_{\text{in}} \notin \mathbf{V}_G(c_{\text{out}})$. Let $c := c_{\text{OUT}}$. By the Validity guarantees of $\mathsf{TreeAA}(T_S)$, there are two honest inputs c_a, c_b in T_S so that c lies on the unique path in T_S, and hence

also in T, connecting c_a and c_b. Then, let c' be the neighbor of c lying on the path from c to c_b. Removing the edge $e = \{c, c'\}$ from T partitions T into two components: T_a (containing c_a and c) and T_b (containing c' and c_b). By Lemma 12 any path in G from the honest inputs v_a and v_b contained by the cliques c_a and c_b must pass through a vertex $u \in \mathsf{V}_G(c) \cap \mathsf{V}_G(c')$. As $\mathsf{V}_G(c) \cap \mathsf{V}_G(c')$ only contains one vertex, $v_{\text{OUT}} = u$. Consequently, v_{OUT} lies on the shortest path between two honest inputs, and therefore is valid. □

We now show that the 1-Agreement property holds. Throughout the following, we use subscripts in the distance notation to distinguish between distances measured in the original graph G and distances measured in the subdivided clique tree T_S.

Lemma 14. *If two honest parties p and p' output v_{OUT} and v'_{OUT} respectively, $\mathsf{d}_G(v_{\text{OUT}}, v'_{\text{OUT}}) \leq 1$.*

Proof. Let c_{OUT} and c'_{OUT} be the outputs that parties p and p' obtain from running $\mathsf{TreeAA}(T_S)$. By the 1-Agreement property of $\mathsf{TreeAA}(T_S)$, we have $\mathsf{d}_{T_S}(c_{\text{OUT}}, c'_{\text{OUT}}) \leq 1$. We split the analysis into two cases.

Case 1: $\mathbf{d}_{T_S}(c_{\text{OUT}}, c'_{\text{OUT}}) = 0$. If c_{OUT} is an intermediate vertex, i.e., a subdivision vertex of some edge $\{c, c'\}$ of T, then both parties output the same vertex. Specifically, $\{v_{\text{OUT}}\} = \{v'_{\text{OUT}}\} = \mathsf{V}_G(c) \cap \mathsf{V}_G(c')$, implying $\mathsf{d}_G(v_{\text{OUT}}, v'_{\text{OUT}}) = 0$. Otherwise, c_{OUT} is a clique vertex. Then, both parties output vertices $v_{\text{OUT}}, v'_{\text{OUT}} \in \mathsf{V}_G(c_{\text{OUT}})$. Since $\mathsf{V}_G(c_{\text{OUT}})$ induces a clique in G, any two vertices in it are either identical or adjacent; therefore, $\mathsf{d}_G(v_{\text{OUT}}, v'_{\text{OUT}}) \leq 1$.

Case 2: $\mathbf{d}_{T_S}(c_{\text{OUT}}, c'_{\text{OUT}}) = 1$. By the definition of the subdivided clique tree T_S, exactly one of c_{OUT} and c'_{OUT} is a clique vertex, and the other is an intermediate vertex. Without loss of generality, assume that c_{OUT} is a clique vertex and c'_{OUT} is the intermediate vertex subdividing the edge $\{c_{\text{OUT}}, c'\}$ of T, for some neighbor c' of c_{OUT}. By the output rule for intermediate vertices, the party corresponding to c'_{OUT} outputs the unique vertex $v'_{\text{OUT}} \in V_G(c_{\text{OUT}}) \cap V_G(c')$. In particular, $v'_{\text{OUT}} \in V_G(c_{\text{OUT}})$. Moreover, by the output rule for clique vertices, the party corresponding to c_{OUT} outputs a vertex $v_{\text{OUT}} \in V_G(c_{\text{OUT}})$. Since $V_G(c_{\text{OUT}})$ induces a clique in G, any two vertices in this set are either identical or adjacent. Therefore, $\mathsf{d}_G(v_{\text{OUT}}, v'_{\text{OUT}}) \leq 1$. □

The lemma below will enable us to prove the round complexity of $\mathsf{BlockGraphAA}(G)$.

Lemma 15. *Let T be a minimum-diameter clique tree of a block graph G. Then $D(T) \leq 2 \cdot D(G)$.*

Proof. If G consists of a single maximal clique, then its clique tree T consists of a single vertex, and hence $\mathsf{D}(T) = 0 \leq 2 \cdot \mathsf{D}(G)$. If G contains exactly two maximal cliques, then T consists of a single edge and thus $\mathsf{D}(T) = 1 \leq 2 \cdot \mathsf{D}(G)$. For the remainder of the proof, we assume that G contains at least three maximal cliques. In particular, $\mathsf{D}(G) \geq 2$.

Let $P := (c_0, c_1, \ldots, c_k)$ be a longest path in T, so that $k = \mathsf{D}(T)$. If $k \leq 2$, then $\mathsf{D}(T) \leq 2 \leq 2 \cdot \mathsf{D}(G)$, and the claim follows. Hence, we assume $k \geq 3$.

For every $i \in \{0, 1, \ldots, k-1\}$, since c_i and c_{i+1} are adjacent in T, we have $\mathsf{V}_G(c_i) \cap \mathsf{V}_G(c_{i+1}) \neq \varnothing$. As G is a block graph, any two distinct maximal cliques intersect in at most one vertex. Therefore, for each i, there exists a unique vertex $v_i \in \mathsf{V}_G(c_i) \cap \mathsf{V}_G(c_{i+1})$. We then define the vertex sequence $S := (v_0, v_1, \ldots, v_{k-1})$. We claim that none of the following cases can occur in the sequence S:

1. There exists an index i such that $\{v_i, v_{i+1}\} \notin E(G)$ and $\{v_{i+1}, v_{i+2}\} \notin E(G)$.
2. There exist indices $0 < i + 1 < j \leq k - 1$ such that $v_i = v_j$.

We first rule out case (1). Since both v_i and v_{i+1} belong to the clique c_{i+1}, that is, $\{v_i, v_{i+1}\} \subseteq \mathsf{V}_G(c_{i+1})$, we have $v_i = v_{i+1}$. This is because, in a clique, every two distinct vertices are adjacent, and if $\{v_i, v_{i+1}\} \notin E(G)$, this implies that $v_i = v_{i+1}$. Similarly, since both v_{i+1} and v_{i+2} belong to the clique c_{i+2}, i.e., $\{v_i, v_{i+1}\} \subseteq \mathsf{V}_G(c_{i+2})$, we have $v_{i+1} = v_{i+2}$. As a result, we obtain $v = v_i = v_{i+1} = v_{i+2}$.

Consequently, the cliques c_i, c_{i+1}, c_{i+2}, and c_{i+3} all intersect in the same vertex v, forming a subpath of length three in T whose cliques share a common intersection $\{v\}$. More generally, let $c_{i+1}, \ldots, c_{i+\ell}$ with $\ell > 2$ be a maximal subpath of T such that $\cap_{r=i}^{r=i+\ell} \mathsf{V}_G(c_r) = \{v\}$. Since $k \geq 3$, such a subpath exists with $\ell > 2$. We then replace this subpath by a star centered at c_i, where $c_{i+1}, \ldots, c_{i+\ell}$ become adjacent to c_i, while preserving their original neighbors in T outside the set $\{c_{i+1}, \ldots, c_{i+\ell}\}$. The running intersection property is preserved because all these cliques contain the common vertex v, and for any two cliques among $c_i, \ldots, c_{i+\ell}$ the unique clique on the path between them in the modified tree is c_i, which also contains v. Hence, every clique on the path between any two cliques containing v continues to contain v. This transformation strictly decreases the diameter of T, contradicting the assumption that T is a minimum-diameter clique tree of G. Hence, $v_i \neq v_{i+1}$ for all i for which $v_{i+1} = v_{i+2}$. Thus, case (1) cannot occur. This implies that in the sequence S no three consecutive vertices can be identical.

We now rule out case (2). Suppose that there exist indices i and j with $i+1 < j$ such that $v_i = v_j = v$. Then $v \in \mathsf{V}_G(c_i) \cap \mathsf{V}_G(c_{i+1})$ and $v \in \mathsf{V}_G(c_j) \cap \mathsf{V}_G(c_{j+1})$. The running intersection property of clique trees implies that v belongs to every clique on the path from c_i to c_{j+1} in T. Let us first consider the case where $v_i \neq v_{i+1}$. Since both v_i and v_{i+1} belong to c_{i+1} and c_{i+2}, we have that $\{v_i, v_{i+1}\} \subseteq \mathsf{V}_G(c_{i+1}) \cap \mathsf{V}_G(c_{i+2})$. This implies that $|\mathsf{V}_G(c_{i+1}) \cap \mathsf{V}_G(c_{i+2})| \geq 2$, which contradicts the assumption that G is a block graph, as in a block graph any two distinct maximal cliques intersect in at most one vertex.

It remains to consider the case where $v_i = v_{i+1}$. If $v_{i+1} \neq v_{i+2}$, then we may follow the same argument as above: replacing v_i by v_{i+1}, we obtain that both v_{i+1} and v_{i+2} belong to c_{i+2} and c_{i+3}, and hence $\{v_{i+1}, v_{i+2}\} \subseteq \mathsf{V}_G(c_{i+2}) \cap \mathsf{V}_G(c_{i+3})$, which again implies that $|\mathsf{V}_G(c_{i+2}) \cap \mathsf{V}_G(c_{i+3})| \geq 2$. This contradicts the assumption that G is a block graph. On the other hand, if $v_{i+1} = v_{i+2}$, then this is exactly case (1), which we have already shown cannot occur. Therefore, there exist no indices $0 < i + 1 < j \leq k - 1$ such that $v_i = v_j$.

Let us construct a sequence P_G in G using S. For every $i \in \{0, 1, \ldots, k-1\}$, if $v_i \neq v_{i+1}$, we add v_i to P_G, and otherwise we skip it. Note that if $v = v_i = v_{i+1}$ and we skip v_i, then by the conclusion of case (1) $v_{i+1} \neq v_{i+2}$, and therefore $v_{i+1} = v$ will be added to P_G. We claim that P_G is a path in G. This is because for every index i with $v_i \neq v_{i+1}$, the vertices v_i and v_{i+1} both belong to a maximal clique, and hence they are adjacent in G, i.e., $\{v_i, v_{i+1}\} \in E(G)$. It follows that consecutive vertices in P_G are adjacent in G. Moreover, by Case (2), the sequence S contains no two vertices v_i and v_j with $i + 1 < j$ such that $v_i = v_j$, and hence the sequence P_G contains no repeated vertex. Therefore, P_G forms a path in G. By the construction of P_G, each vertex of P_G can appear at most twice in S, and hence $|S| \leq 2 \cdot |P_G|$. In particular, since $|S| = k - 1 = \mathsf{D}(T) - 1$, we obtain $\mathsf{D}(T) \leq 2 \cdot |P_G|$.

Since P_G is a path in G, it follows that $|P_G| \leq \mathsf{D}(G)$. Combining this with $\mathsf{D}(T) \leq 2 \cdot |P_G|$, we obtain $\mathsf{D}(T) \leq 2 \cdot \mathsf{D}(G)$. □

We now present the proof of Theorem 8.

Proof of Theorem 8. To compute T_S, the parties use the deterministic algorithm presented in [11], which provides them with the same minimum-diameter clique tree T of G. Afterwards, they execute $\mathsf{TreeAA}(T_S)$. Then, the parties complete the execution of $\mathsf{TreeAA}(T_S)$ within $R_{\mathsf{TreeAA}}(\mathsf{D}(T_S))$ rounds. Therefore, they obtain outputs in $\mathsf{BlockGraphAA}(G)$ within $R_{\mathsf{TreeAA}}(\mathsf{D}(T_S))$ rounds, hence Termination holds. As $\mathsf{D}(T_S) \leq 2 \cdot \mathsf{D}(T)$ by construction, and $\mathsf{D}(T) \leq 2 \cdot \mathsf{D}(G)$ by Lemma 15, we obtain that $\mathsf{BlockGraphAA}(G)$ has round complexity $R_{\mathsf{TreeAA}}(4 \cdot \mathsf{D}(G))$, as claimed. Lemma 13 and Lemma 14 guarantee that Validity and 1-Agreement hold, concluding the proof. □

We obtain our final corollaries for block graphs by instantiating $\mathsf{TreeAA}(T_S)$ with the protocol described in Corollary 2 in unauthenticated settings (with no cryptographic assumptions), and with the protocol of Corollary 3 for authenticated settings (assuming PKI and digital signatures). Note that, by Theorem 1, we have obtained asymptotically optimal round complexity whenever $t \in \Theta(n)$.

Corollary 4. *There is a synchronous protocol that achieves $\mathcal{AA}$ on any connected block graph G in $O\left(\frac{\log \mathsf{D}(G)}{\log\log \mathsf{D}(G)}\right)$ rounds given that up to $t < n/3$ of the n parties involved are Byzantine.*

Corollary 5. *There is a synchronous protocol that achieves $\mathcal{AA}$ on any connected block graph G in $O\left(\frac{\log \mathsf{D}(G)}{\log\log \mathsf{D}(G)}\right)$ rounds in authenticated settings given that up to $t = c \cdot n$ of the n parties are Byzantine, where c is a constant in $[1/3, 1/2)$.*

Moreover, we note that $\mathsf{BlockGraphAA}(G)$ does not depend on the network being synchronous – it achieves $\mathcal{AA}$ on G whenever the underlying $\mathsf{TreeAA}(T_S)$ protocol achieves $\mathcal{AA}$ on the subdivided clique tree T_S. By instantiating $\mathsf{TreeAA}(T_S)$ with the asynchronous protocol of [37, Theorem 5], we obtain another corollary, for the asynchronous model. We recall that the asynchronous model only assumes that messages get delivered eventually.

Corollary 6. *There is an asynchronous protocol that solves $\mathcal{AA}$ on any connected block graph G given that up to $t < n/3$ of the n parties involved are Byzantine.*

6 Conclusions

In this work, we have investigated the optimal round complexity for achieving $\mathcal{AA}$ on trees in the synchronous model. Our results extend previous findings from real-valued domains to tree-structured input spaces, providing novel insights for achieving efficient $\mathcal{AA}$ in discrete input spaces. We presented a protocol with round complexity $O\left(\frac{\log \mathsf{D}(T)}{\log\log \mathsf{D}(T)}\right)$, improving upon the previously best-known $O(\log \mathsf{D}(T))$-round protocol of [37]. Our construction achieves optimal resilience and relies on a reduction from $\mathcal{AA}$ on trees to its real-valued counterpart. Moreover, we have provided an asymptotically matching lower bound by adapting the lower bound of [20] from real values to graphs, demonstrating that our protocol achieves asymptotically optimal round complexity whenever $t \in \Theta(n)$.

We further extend our techniques to block graphs by exploiting their clique tree structure. By reducing $\mathcal{AA}$ on a block graph to $\mathcal{AA}$ on a suitably constructed tree, we obtain efficient synchronous and asynchronous protocols with optimal resilience, with asymptotically optimal round complexity in the synchronous model whenever $t \in \Theta(n)$.

In contrast to prior work on $\mathcal{AA}$ outside the real line, our protocols rely only on efficient local computation and avoid the exponential-time safe-area constructions.

Our results suggest several directions for further investigation. On the practical side, improving the constants in the round complexity (particularly those inherited from real-valued $\mathcal{AA}$) would directly improve the efficiency of our protocols for trees and block graphs. On the theoretical side, it remains open whether similarly optimal-resilience efficient protocols can be obtained for broader classes of graphs, and where the precise structural boundary lies between $\mathcal{AA}$ and $\mathcal{CA}$ in terms of resilience. More broadly, we view reduction-based techniques such as ours as a promising approach for efficient $\mathcal{AA}$ across diverse input spaces.

References

[1] Ittai Abraham, Yonatan Amit, and Danny Dolev. Optimal resilience asynchronous approximate agreement. In Teruo Higashino, editor, *Principles of Distributed Systems*, pages 229–239, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[2] Manuel Alcántara, Armando Castañeda, David Flores-Peñaloza, and Sergio Rajsbaum. The topology of look-compute-move robot wait-free algorithms with hard termination. *Distributed Computing*, 32(3):235–255, 2019.

[3] Dan Alistarh, Faith Ellen, and Joel Rybicki. Wait-free approximate agreement on graphs. In Tomasz Jurdziński and Stefan Schmid, editors, *Structural Information and Communication Complexity*, pages 87–105, Cham, 2021. Springer International Publishing.

[4] Hagit Attiya, Itay Flam, and Jennifer L. Welch. Beyond Canonical Rounds: Communication Abstractions for Optimal Byzantine Resilience, 2025.

[5] Hagit Attiya and Jennifer L. Welch. Multi-Valued Connected Consensus: A New Perspective on Crusader Agreement and Adopt-Commit. In Alysson Bessani, Xavier Défago, Junya Nakamura, Koichi Wada, and Yukiko Yamauchi, editors, *27th International Conference on Principles of Distributed Systems (OPODIS 2023)*, volume 286 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 6:1–6:23, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[6] A. Bandarupalli, A. Bhat, S. Bagchi, A. Kate, C.-D. Liu-Zhang, and M. K. Reiter. Delphi: Efficient asynchronous approximate agreement for distributed oracles. In *Proceedings of the 2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 456–469, Brisbane, Australia, 2024. IEEE.

[7] Michael Ben-Or, Danny Dolev, and Ezra N. Hoch. Brief announcement: Simple gradecast based algorithms. In Nancy A. Lynch and Alexander A. Shvartsman, editors, *Distributed Computing*, pages 194–197, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.

[8] Michael Ben-Or, Danny Dolev, and Ezra N. Hoch. Simple gradecast based algorithms, 2010.

[9] Michael A. Bender and Martín Farach-Colton. The lca problem revisited. In Gaston H. Gonnet and Alfredo Viola, editors, *LATIN 2000: Theoretical Informatics*, pages 88–94, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.

[10] Anne Berry and Romain Pogorelcnik. A simple algorithm to generate the minimal separators and the maximal cliques of a chordal graph. *Information Processing Letters*, 111(11):508–511, 2011.

[11] Jean R. S. Blair and Barry W. Peyton. On finding minimum-diameter clique trees. *Nordic J. of Computing*, 1(2):173–201, June 1994.

[12] B.A. Coan. A compiler that increases the fault tolerance of asynchronous protocols. *IEEE Transactions on Computers*, 37(12):1541–1553, 1988.

[13] Andrei Constantinescu, Diana Ghinea, Roger Wattenhofer, and Floris Westermann. Convex Consensus with Asynchronous Fallback. In Dan Alistarh, editor, *38th International Symposium on Distributed Computing (DISC 2024)*, volume 319 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 15:1–15:23, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[14] Gianlorenzo D'Angelo, Gabriele Di Stefano, and Alfredo Navarra. Gathering on rings under the look-compute-move model. *Distributed Computing*, 27(4):255–285, 2014.

[15] Danny Dolev, Nancy A. Lynch, Shlomit S. Pinter, Eugene W. Stark, and William E. Weihl. Reaching approximate agreement in the presence of faults. *J. ACM*, 33(3):499–516, May 1986.

[16] Danny Dolev and H. Raymond Strong. Authenticated algorithms for byzantine agreement. *SIAM Journal on Computing*, 12(4):656–666, 1983.

[17] El-Mahdi El-Mhamdi, Sadegh Farhadkhani, Rachid Guerraoui, Arsany Guirguis, Lê-Nguyên Hoang, and Sébastien Rouault. Collaborative learning in the jungle (decentralized, byzantine, heterogeneous, asynchronous and nonconvex learning). In *Proceedings of the 35th International Conference on Neural Information Processing Systems*, NIPS '21, Red Hook, NY, USA, 2021. Curran Associates Inc.

[18] El-Mahdi El-Mhamdi, Rachid Guerraoui, Arsany Guirguis, Lê Nguyên Hoang, and Sébastien Rouault. Genuinely distributed byzantine machine learning. In *Proceedings of the 39th Symposium on Principles of Distributed Computing*, PODC '20, page 355–364, New York, NY, USA, 2020. Association for Computing Machinery.

[19] A. D. Fekete. Asynchronous approximate agreement. In *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing*, PODC '87, page 64–76, New York, NY, USA, 1987. Association for Computing Machinery.

[20] Alan David Fekete. Asymptotically optimal algorithms for approximate agreement. *Distributed Computing*, 4(1):9–29, 1990.

[21] Paola Flocchini, Giuseppe Prencipe, Nicola Santoro, and Peter Widmayer. Gathering asynchronous oblivious agents with local vision in regular bipartite graphs. *Theoretical Computer Science*, 509:26–41, 2013.

[22] Marc Fuchs, Diana Ghinea, and Zahra Parsaeian. Brief announcement: Towards round-optimal approximate agreement on trees. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*, PODC '25, page 54–57, New York, NY, USA, 2025. Association for Computing Machinery.

[23] Diana Ghinea, Vipul Goyal, and Chen-Da Liu-Zhang. Round-optimal byzantine agreement. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022*, pages 96–119, Cham, 2022. Springer International Publishing.

[24] Diana Ghinea and Chen-Da Liu-Zhang. SoK: Approximate Agreement. In *Financial Cryptography and Data Security 2026 (FC), St. Kitts*, March 2026.

[25] Diana Ghinea, Chen-Da Liu-Zhang, and Roger Wattenhofer. Optimal synchronous approximate agreement with asynchronous fallback. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing*, PODC'22, page 70–80, New York, NY, USA, 2022. Association for Computing Machinery.

[26] Diana Ghinea, Chen-Da Liu-Zhang, and Roger Wattenhofer. Multidimensional approximate agreement with asynchronous fallback. In *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA '23, page 141–151, New York, NY, USA, 2023. Association for Computing Machinery.

[27] Diana Ghinea, Chen-Da Liu-Zhang, and Roger Wattenhofer. Brief announcement: Communication-optimal convex agreement. In *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing*, PODC '24, page 492–495, New York, NY, USA, 2024. Association for Computing Machinery.

[28] Diana Ghinea, Chen-Da Liu-Zhang, and Roger Wattenhofer. Communication-optimal convex agreement. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*, PODC '25, page 39–49, New York, NY, USA, 2025. Association for Computing Machinery.

[29] Jérémy Ledent. Brief announcement: Variants of approximate agreement on graphs and simplicial complexes. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, PODC'21, page 427–430, New York, NY, USA, 2021. Association for Computing Machinery.

[30] Christoph Lenzen and Julian Loss. Optimal clock synchronization with signatures. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing*, PODC'22, page 440–449, New York, NY, USA, 2022. Association for Computing Machinery.

[31] Shihao Liu. The Impossibility of Approximate Agreement on a Larger Class of Graphs. In Eshcar Hillel, Roberto Palmieri, and Etienne Rivière, editors, *26th International Conference on Principles of Distributed Systems (OPODIS 2022)*, volume 253 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 22:1–22:20, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[32] Darya Melnyk and Roger Wattenhofer. Byzantine agreement with interval validity. In *2018 IEEE 37th Symposium on Reliable Distributed Systems (SRDS)*, pages 251–260, Salvador, Brazil, 2018. IEEE Computer Society.

[33] Hammurabi Mendes and Maurice Herlihy. Multidimensional approximate agreement in byzantine asynchronous systems. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 391–400, Palo Alto, CA, USA, June 2013. ACM Press.

[34] Hammurabi Mendes, Maurice Herlihy, Nitin Vaidya, and Vijay K Garg. Multidimensional agreement in byzantine systems. *Distributed Computing*, 28(6):423–441, 2015.

[35] Silvio Micali and Vinod Vaikuntanathan. Optimal and player-replaceable consensus with an honest majority, 2017.

[36] Mose Mizrahi Erbes and Roger Wattenhofer. Asynchronous Approximate Agreement with Quadratic Communication. In Andrei Arusoaie, Emanuel Onica, Michael Spear, and Sara Tucci-Piergiovanni, editors, *29th International Conference on Principles of Distributed Systems (OPODIS 2025)*, volume 361 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 16:1–16:26, Dagstuhl, Germany, 2026. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[37] Thomas Nowak and Joel Rybicki. Byzantine Approximate Agreement on Graphs. In Jukka Suomela, editor, *33rd International Symposium on Distributed Computing (DISC 2019)*, volume 146 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 29:1–29:17, Dagstuhl, Germany, 2019. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.

[38] Maria Potop-Butucaru, Michel Raynal, and Sebastien Tixeuil. Distributed computing with mobile robots: An introductory survey. In *Proceedings of the 2011 14th International Conference on Network-Based Information Systems*, NBIS '11, page 318–324, USA, 2011. IEEE Computer Society.

[39] David Stolz and Roger Wattenhofer. Byzantine Agreement with Median Validity. In Emmanuelle Anceaume, Christian Cachin, and Maria Potop-Butucaru, editors, *19th International Conference on Principles of Distributed Systems (OPODIS 2015)*, volume 46 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 1–14, Dagstuhl, Germany, 2016. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.

[40] Lili Su and Nitin H. Vaidya. Fault-tolerant multi-agent optimization: Optimal iterative distributed algorithms. In *Proceedings of the 2016 ACM Symposium on Principles of Distributed Computing*, PODC '16, page 425–434, New York, NY, USA, 2016. Association for Computing Machinery.

[41] Nitin H. Vaidya and Vijay K. Garg. Byzantine vector consensus in complete graphs. In Panagiota Fatourou and Gadi Taubenfeld, editors, *32nd ACM PODC*, pages 65–73, Montreal, QC, July 2013. ACM.

Appendix

A Protocol for trees: Additional Proofs

We include the proof of the second part of Remark 1.

Proof. Assume without loss of generality that $j < j'$. Since $j' - j \leq 1$, either there is an integer z such that $z \leq j, j', \leq z + 1$ or there is an integer z such that $z - 1 \leq j \leq z \leq j' \leq z + 1$. In the first case, both $\lfloor j \rfloor$ and $\lfloor j' \rfloor$ are either z or $z + 1$ by the definition of $\lfloor \cdot \rfloor$. In the second case, assume that $\lfloor j \rfloor = z - 1$, and $\lfloor j' \rfloor = z + 1$. Then, $j - (z - 1) < z - j$, hence $j < z - \frac{1}{2}$, while $(z + 1) - j' \leq j' - z$, and therefore $j' \geq z + \frac{1}{2}$. This leads to a contradiction, as it implies $j' - j > 1$. Hence, in both cases, $\lfloor j' \rfloor - \lfloor j \rfloor \leq 1$. □

B RealAA in the $t < n/3$ setting

As mentioned in Section 4.1, the analysis of [7] regarding RealAA proves that $\mathcal{AA}$ is achieved for $\varepsilon = 1/n$. Theorem 2 extends the analysis for any $\varepsilon > 0$.

In order to prove Theorem 2, we make use of a few results in the full version [8] of [7]. We first need to note the round complexity of each iteration in RealAA. This remark follows directly from [7, Theorem 1], which discusses the guarantees of the mechanism the parties use to distribute their current values in each iteration.

Remark 3 (Theorem 1 of [8])**.** *Each iteration of* $\mathsf{RealAA}(\varepsilon)$ *takes three rounds.*

Our proof also utilizes the two claims stated below: these are concerned with ε-Agreement and Validity. In the following, V_R denotes the multiset of values held by the honest parties at the end of iteration R (hence, round $3 \cdot R$), and V_0 denotes the multiset of honest inputs.

Lemma 16 (Claim 12 of [8])**.** *In every iteration* R, $(\max V_R - \min V_R) \leq (\max V_0 - \min V_0) \cdot \frac{t^R}{R^R \cdot (n-2t)^R}$.

Lemma 17 (Claim 8 of [8])**.** *In every iteration* R, $V_R \subseteq [\min V_0, \max V_0]$.

We are now ready to prove Theorem 2, which we restate below.

Theorem 2. *There is a protocol* $\mathsf{RealAA}(\varepsilon)$ *achieving* $\mathcal{AA}$ *on real values even when up to* $t < n/3$ *of the* n *parties involved are Byzantine. If the honest inputs are* D*-close,* $\mathsf{RealAA}(\varepsilon)$ *ensures Termination within* $R_{\mathsf{RealAA}}(\mathsf{D}, \varepsilon) < 7 \cdot \frac{\log_2(\mathsf{D}/\varepsilon)}{\log_2 \log_2(\mathsf{D}/\varepsilon)} + 3$ *rounds.*

Proof. By the Lemma 16, we know that the maximum absolute difference between the output values of any two honest parties is at most $\frac{\mathsf{D} \cdot t^R}{(n-2t)^R \cdot R^R}$ after R iterations. Since $n > 3t$, this is at most $\frac{\mathsf{D}}{R^R}$. We will now show that for $R := \lceil \frac{20}{9} \cdot \frac{\log \delta}{\log \log \delta} \rceil$, where $\delta := \mathsf{D}/\varepsilon$, we obtain $\frac{\mathsf{D}}{R^R} \leq \varepsilon$ and thus ε-Agreement holds after at most R iterations. Below we provide a lower bound for $\log_2(R^R)$:

$$
\begin{aligned}
\log_2(R^R) &= R \log_2 R \\
&\geq \frac{20}{9} \cdot \frac{\log_2 \delta}{\log_2 \log_2 \delta} \cdot (\log_2 \log_2 \delta - \log_2 \log_2 \log_2 \delta) \\
&\geq \frac{20}{9} \cdot \frac{\log_2 \delta}{\log_2 \log_2 \delta} \cdot (\log_2 \log_2 \delta - \frac{11}{20} \cdot \log_2 \log_2 \delta) \\
&= \log_2 \delta \\
&= \log_2 \left(\frac{\mathsf{D}}{\varepsilon} \right).
\end{aligned}
$$

By exponentiation on both sides (with base 2), we obtain that $R^R \geq \frac{\mathsf{D}}{\varepsilon}$, and therefore our claim $\frac{\mathsf{D}}{R^R} \leq \varepsilon$ holds, which proves that ε-Agreement is reached. Note that, in the third line, we use the fact that since the function $\log_2(x)/x$, defined for $x > 0$, has a global maximum (at $x = e$) of $\frac{1}{e \cdot \ln(2)} < 11/20$ and therefore also $\log_2(\log_2 \log_2(x))/\log_2(\log_2(x)) < 11/20$ for all $x > 0$.

As each of the iterations of RealAA requires three rounds according to Remark 3, the protocol terminates after at most $R_{\mathsf{RealAA}}(\mathsf{D}, \varepsilon) \leq 3 \cdot R < 7 \cdot \frac{\log_2(\mathsf{D}/\varepsilon)}{\log_2 \log_2(\mathsf{D}/\varepsilon)} + 3$ rounds. In addition, Validity holds due to Lemma 17, hence $\mathcal{AA}$ is achieved. □

Below we present the proof of Lemma 2.

Lemma 2. *Consider a set of vertices $S \subseteq \mathsf{V}(T)$ and a path P in T such that $\mathsf{V}(P) \cap \langle S \rangle \neq \varnothing$. Then, for any $v \in S$, $\mathsf{proj}_P(v) \in \mathsf{V}(P) \cap \langle S \rangle$.*

Proof. Let v be an arbitrary vertex in S. If $v \in \mathsf{V}(P)$, then $v = \mathsf{proj}_P(v)$ and the statement follows immediately. The remainder of the proof is therefore concerned with the case where $v \notin \mathsf{V}(P)$.

Let v_h be a vertex in $\mathsf{V}(P) \cap \langle S \rangle$. Note that all vertices on the path $P(v, v_h)$ are in $\langle S \rangle$ since both v and v_h are in $\langle S \rangle$. We label the vertices in $P(v, v_h)$ as $(v_1 := v, v_2, \ldots, v_m := v_h)$. As we know that v_1 is not on P but v_h is on P, there is an i satisfying $1 < i \leq h$ such that $v_i \in \mathsf{V}(P)$ and $v_1, \ldots, v_{i-1} \notin \mathsf{V}(P)$.

Then, v_i must be the projection of v onto P, i.e., $v_i = \mathsf{proj}_P(v)$. Assuming otherwise implies that either there is another vertex within $v_1, \ldots, v_{i-1}$ that is in $\mathsf{V}(P)$, which contradicts the way we have defined i, or that the tree T contains a cycle. Therefore, because $v_i \in \mathsf{V}(P)$ is the projection of v onto P, it follows by our previous observation $v_i \in \langle S \rangle$ that $\mathsf{proj}_P(v) \in \mathsf{V}(P) \cap \langle S \rangle$. □

C RealAA in the $t < n/2$ setting

In this section, we focus on $\mathcal{AA}$ in the authenticated setting, where we can handle up to $t < n/2$ Byzantine parties. Theorem 2 already handles the range $t < n/3$, hence in the following we only focus on the remaining range, i.e., t satisfying $n/3 \leq t < n/2$. We discuss the proof of Theorem 7, restated below.

Theorem 7. *If the honest inputs are D-close real values, there is a protocol $\mathsf{RealAA}^{\mathsf{PKI}}(\mathsf{D}, \varepsilon)$ achieving $\mathcal{AA}$ in authenticated settings even when up to $t = c \cdot n$ (where c is a constant in $[1/3, 1/2)$) of the n parties are Byzantine. $\mathsf{RealAA}^{\mathsf{PKI}}(\mathsf{D}, \varepsilon)$ achieves Termination within $R_{\mathsf{RealAA}^{\mathsf{PKI}}}(\mathsf{D}, \varepsilon) = O\left(\frac{\log(\mathsf{D}/\varepsilon)}{\log\log(\mathsf{D}/\varepsilon)}\right)$ rounds.*

As mentioned in the main body of the paper, we obtain this result by combining two protocols: the Proxcensus protocol of [23], which will be used when D/ε is large, and the $\mathcal{AA}$ protocol of [25], which will be used otherwise.

When D/ε is large. As previously mentioned, we rely on the protocol of [23]. This protocol implements, in fact, *Binary Proxcensus* – the parties join with inputs in $\{0, 1\}$. For a given integer $\ell \geq 1$, the parties output 1-close integer values in $\{0, 1, \ldots, \ell\}$ such that: if all honest parties have input 0, all honest parties output 0, and if all honest parties have input 1, all honest parties output 1. In the protocol of [23], the parties first *expand* their inputs from $\{0, 1\}$ to $\{0, \ell\}$: input 0 remains 0, and input 1 becomes ℓ. Afterwards, the parties run a series of iterations where they compute new values with the guarantee that (i) honest parties' new values are in the range of the honest parties' values at the beginning of the iteration, (ii) the range of honest values is reduced with each iteration. We note that the new values are computed as a *rounded* average (so that parties hold integer values in each iteration), but removing the rounding maintains this guarantee. Then, this is essentially an $\mathcal{AA}$ protocol on $[0, \ell]$: the only adjustments that we need to make are (i) allowing parties to join directly with inputs from a bounded interval instead of mapping binary inputs to $\{0, \ell\}$, (ii) removing the rounding in the new values' computation, and (iii) fixing the number of iterations based on our use-case. We will see that, if honest parties hold D-close

inputs and we want their outputs to be ε-close, $O\left(\frac{\log(D/\varepsilon)}{\log\log(D/\varepsilon)}\right)$ iterations suffice whenever $t = c \cdot n$ for some constant $c \in [1/3, 1/2)$. More precisely, $\lceil \frac{20}{9 \cdot c'} \cdot \frac{\log_2 \delta}{\log_2 \log_2 \delta} \rceil$ iterations, where $c' := \frac{1-2c}{c}$ and $\delta := \left(\mathsf{D}/\varepsilon\right)^{c'}$, will suffice. In the following, we refer to the adjusted protocol as $\mathsf{RealAA}_1^{\mathsf{PKI}}$, and we prove the following statement.

Theorem 9. *If the honest inputs are D-close real values, there is a protocol $\mathsf{RealAA}_1^{PKI}(D, \varepsilon)$ achieving $\mathcal{AA}$ even when up to $t = c \cdot n$ (where c is a constant in $[1/3, 1/2)$) of the n parties are Byzantine, ensuring termination within*

$$R_{\mathsf{RealAA}_1^{PKI}(D,\varepsilon)} < 7 \cdot \frac{\log_2(D/\varepsilon)}{\log_2\left((1-2c)/c\right) + \log_2 \log_2(D/\varepsilon)} + 3$$

rounds.

Note that, $\log_2\left((1-2c)/c\right)$ is non-positive for our chosen range of $c \in [1/3, 1/2)$. However, the denominator is non-positive only when D/ε is a constant – in this case, $\mathsf{RealAA}^{\mathsf{PKI}}$ will make use of the protocol of [25] instead: this protocol achieves Termination within $O(\mathsf{D}/\varepsilon) = O(1)$ rounds.

We first note that each iteration requires three rounds of communication, as stated in [23].

Remark 4. *Each iteration of $\mathsf{RealAA}_1^{PKI}(\varepsilon)$ takes three rounds.*

Then V_R denotes the multiset of values obtained by the honest parties in iteration R, and V_0 denotes the multiset of honest inputs. The lemma below establishes that Validity is maintained at all times.

Lemma 18 (Lemma 4 of [23])**.** *After R iterations, $V_R \subseteq [\min V_0, \max V_0]$.*

The following lemma follows from Lemma 6 of [23]. We note that this lemma highlights the need for the constant $c < 1/2$, as setting $n := 2t+1$ instead would mean that the honest values are not guaranteed to converge unless $R > t$.

Lemma 19 (Lemma 6 in [23])**.** *After R iterations,* $\max V_R - \min V_R \leq (\max V_0 - \min V_0) \cdot \frac{t^R}{R^R \cdot (n-2t)^R}$.

The lemma below discusses the number of iterations that are sufficient for ε-Agreement to be achieved. Recall that $t = c \cdot n$, where $c \in [1/3, 1/2)$ is a constant, and an upper bound on the honest inputs' range size D, i.e., $\mathsf{D} \geq \max V_0 - \min V_0$, is assumed to be known. The proof will be similar to that of RealAA presented in the proof of Theorem 2.

Lemma 20. *Let $R := \lceil \frac{20}{9 \cdot c'} \cdot \frac{\log_2 \delta}{\log_2 \log_2 \delta} \rceil$, where $\delta := \left(D/\varepsilon\right)^{c'}$ and $c' := \frac{1-2c}{c}$. Then,*

$$\max V_R - \min V_R \leq \varepsilon.$$

Proof. By Lemma 19 we have the upper bound upper bound $\max V_R - \min V_R \leq \mathsf{D} \cdot \frac{t^R}{(n-2t)^R \cdot R^R}$. As $t = c \cdot n$, we can make the following simplification: $\frac{t^R}{(n-2t)^R} \leq \frac{(c \cdot n)^R}{((1-2c) \cdot n)^R} = \frac{c^R}{(1-2c)^R}$.

Let $c' := \frac{1-2c}{c}$. This yields that $\frac{\mathsf{D} \cdot t^R}{(n-2t)^R \cdot R^R}$ is at most $\frac{\mathsf{D}}{c'^R R^R}$. We claim that choosing $R := \lceil \frac{20}{9 \cdot c'} \cdot \frac{\log_2 \delta}{\log_2 \log_2 \delta} \rceil$, where $\delta := \left(\mathsf{D}/\varepsilon\right)^{c'}$, leads to $\frac{\mathsf{D}}{c'^R R^R} \leq \varepsilon$ and thus ε-Agreement holds after at most R iterations.

Below we provide a lower bound for $\log_2\left((c' \cdot R)^R\right)$.

$$\begin{aligned}
\log_2\left((c' \cdot R)^R\right) &= R \log_2(c' \cdot R) \\
&\geq \frac{1}{c'} \cdot \frac{20}{9} \cdot \frac{\log_2 \delta}{\log_2 \log_2 \delta} \cdot \log_2\left(\frac{20}{9} \cdot \frac{\log_2 \delta}{\log_2 \log_2 \delta}\right) \\
&\geq \frac{1}{c'} \cdot \frac{20}{9} \cdot \frac{\log_2 \delta}{\log_2 \log_2 \delta} \cdot (\log_2 \log_2 \delta - \log_2 \log_2 \log_2 \delta) \\
&\geq \frac{1}{c'} \cdot \frac{20}{9} \cdot \frac{\log_2 \delta}{\log_2 \log_2 \delta} \cdot (\log_2 \log_2 \delta - \frac{11}{20} \cdot \log_2 \log_2 \delta) \\
&\geq \frac{1}{c'} \cdot \log_2 \delta \\
&= \log_2\left(\frac{\mathsf{D}}{\varepsilon}\right).
\end{aligned}$$

By exponentiation on both sides (with base 2), we obtain that $(c' \cdot R)^R \geq \frac{\mathsf{D}}{\varepsilon}$, and therefore our claim follows by the following lines:

$$\max V_R - \min V_R \leq \mathsf{D} \cdot \frac{t^R}{(n-2t)^R \cdot R^R} \leq \frac{\mathsf{D}}{c'^R \cdot R^R} \leq \frac{\varepsilon \cdot c'^R \cdot R^R}{c'^R \cdot R^R} = \varepsilon$$

The other simplifications used here are the same as we used in Theorem 2. □

We may now present the proof of Theorem 9. The round complexity analysis will be similar to that of RealAA presented in the proof of Theorem 2.

Proof of Theorem 9. As $\mathsf{RealAA}_1^{\mathsf{PKI}}$ runs for a fixed number of iterations $R := \lceil \frac{20}{9 \cdot c'} \cdot \frac{\log_2 \delta}{\log_2 \log_2 \delta} \rceil$, where $\delta := \left(\mathsf{D}/\varepsilon\right)^{c'}$ and $c' := \frac{1-2c}{c}$, Termination is guaranteed. Since each of the iterations of $\mathsf{RealAA}_1^{\mathsf{PKI}}$ requires three rounds according to Remark 4, the protocol terminates after at most

$$R_{\mathsf{RealAA}_1^{\mathsf{PKI}}}(\mathsf{D}, \varepsilon) \leq 3R < 7 \cdot \frac{\log_2(\mathsf{D}/\varepsilon)}{\log_2 c' + \log_2 \log_2(\mathsf{D}/\varepsilon)} + 3$$

rounds. As $c = O(1)$, also c' is a constant and hence the asymptotic runtime is as described in the theorem's statement. In addition, Lemma 20 ensures that ε-Agreement holds, and Validity holds due to Lemma 18, hence $\mathcal{AA}$ is achieved. □

When D/ε is small. If $\log_2\left((1-2c)/c\right) + \log_2 \log_2(\mathsf{D}/\varepsilon) \leq 0$ (and hence D/ε is a constant), we make use of the $\mathcal{AA}$ protocol of [25], which we denote by $\mathsf{RealAA}_2^{\mathsf{PKI}}$. This protocol is designed in the *network-agnostic model*: given t_s, t_a such that $t_a \leq t_s$ and $2 \cdot t_s + t_a < n$, $\mathsf{RealAA}_2^{\mathsf{PKI}}$ tolerates up to t_s Byzantine corruptions when running in a synchronous network, and up to t_a corruptions when running in an asynchronous network. By setting $t_a := 0$, this becomes a synchronous $\mathcal{AA}$ protocol resilient against $t = t_s < n/2$ Byzantine corruptions. $\mathsf{RealAA}_2^{\mathsf{PKI}}$ follows the outline described in our paper's introduction, halving the honest values' range with each iteration. Each iteration takes four rounds of communication, hence $\mathcal{AA}$ is achieved within $O(\log(D/\varepsilon))$ rounds, which in this case is a constant number of rounds.

Theorem 10 (Theorem 3.1 of [25])**.** *Given that the honest inputs are D-close real values, there is a protocol $\mathsf{RealAA}_2^{PKI}(D, \varepsilon)$ achieving $\mathcal{AA}$ even when $t < n/2$ of the n parties are Byzantine, ensuring Termination within $R_{\mathsf{RealAA}_2^{PKI}}(D, \varepsilon) = 4 \cdot \lceil \log_2(D/\varepsilon) \rceil$ rounds.*

Putting it all together. We may now provide the proof of Theorem 7, where we describe the protocol $\mathsf{RealAA}^{\mathsf{PKI}}$.

Proof of Theorem 7. In $\mathsf{RealAA}^{\mathsf{PKI}}(\mathsf{D},\varepsilon)$, the parties first check locally whether the condition $\log_2\big((1-2c)/c\big)+\log_2\log_2(\mathsf{D}/\varepsilon) > 0$ holds. If this is the case, the parties run $\mathsf{RealAA}_1^{\mathsf{PKI}}(\mathsf{D},\varepsilon)$, described by Theorem 9. Otherwise, the parties run $\mathsf{RealAA}_2^{\mathsf{PKI}}(\mathsf{D},\varepsilon)$, described by Theorem 10. $\mathcal{AA}$ is achieved either way.

Regarding round complexity, note that, if $\log_2\big((1-2c)/c\big)+\log_2\log_2(\mathsf{D}/\varepsilon) \leq 0$, then D/ε is a constant. This implies that $\mathsf{RealAA}_2^{\mathsf{PKI}}(\mathsf{D},\varepsilon)$ according to Theorem 10, and hence $\mathsf{RealAA}^{\mathsf{PKI}}(\mathsf{D},\varepsilon)$, terminates within $O(1)$ rounds. Otherwise, if $\log_2\big((1-2c)/c\big)+\log_2\log_2(\mathsf{D}/\varepsilon) > 0$, Theorem 9 ensures that $\mathsf{RealAA}_1^{\mathsf{PKI}}(\mathsf{D},\varepsilon)$, and therefore $\mathsf{RealAA}^{\mathsf{PKI}}(\mathsf{D},\varepsilon)$ terminates within $O\left(\frac{\log_2(\mathsf{D}/\varepsilon)}{\log_2\log_2(\mathsf{D}/\varepsilon)}\right)$ rounds. □

D Proof of Lemma 12

In this section, we present the proof of Lemma 12. For completeness, we first restate the lemma and then provide its proof.

Lemma 12 (Clique-Tree Edge Separator)**.** *Let G be a connected block graph and let T be a clique tree of G. Let $\{c,c'\}$ be an edge of T and let $u \in \mathsf{V}_G(c) \cap \mathsf{V}_G(c')$. Removing the edge $\{c,c'\}$ partitions T into two connected components T_c and $T_{c'}$ containing c and c', respectively. Let $V_c := \bigcup_{v\in\mathsf{V}(T_c)} \mathsf{V}_G(v)$ and $V_{c'} := \bigcup_{v\in\mathsf{V}(T_{c'})} \mathsf{V}_G(v)$. Then for any $x \in V_c \setminus \{u\}$ and any $y \in V_{c'} \setminus \{u\}$, every path in G from x to y passes through u. Consequently, u is a cut vertex of G.*

Proof. We first note that $\{u\} = \mathsf{V}_G(c) \cap \mathsf{V}_G(c')$: c and c' represent maximal cliques by the definition of clique trees and, by the definition of block graphs, every two maximal cliques intersect in at most one vertex. Assume for contradiction that there exists an x–y path $P := (v_0 = x, v_1, \ldots, v_m = y)$ in G such that $u \notin \{v_0,\ldots,v_m\}$. Without loss of generality, assume that P is a shortest such path.

For each $i \in \{0,\ldots,m-1\}$, let c_i be a maximal clique of G that contains the edge $\{v_i, v_{i+1}\}$.[1] Then for every $i \in \{1,\ldots,m-1\}$, $v_i \in \mathsf{V}_G(c_{i-1}) \cap \mathsf{V}_G(c_i)$, and hence $\mathsf{V}_G(c_{i-1}) \cap \mathsf{V}_G(c_i) \neq \varnothing$. By the running intersection property of clique trees, every clique on the unique path in T between c_{i-1} and c_i also contains vertex v_i. Therefore, the union of all these clique-tree paths (over all indices i) forms a connected subtree T_P of T that contains all cliques c_0, c_1, …, c_{m-1}.

Since $x \in V_c$ and $y \in V_{c'}$, the subtree T_P intersects both components T_c and $T_{c'}$ of $T \setminus \{\{c,c'\}\}$. Therefore, T_P must contain two adjacent clique vertices $d \in \mathsf{V}(T_c)$ and $d' \in \mathsf{V}(T_{c'})$ joined by the edge $\{c,c'\}$.

Since d and d' are adjacent in the clique tree, their corresponding cliques intersect. By the running intersection property, we have that $\mathsf{V}_G(d) \cap \mathsf{V}_G(d') \subseteq \mathsf{V}_G(c) \cap \mathsf{V}_G(c') = \{u\}$. Hence $\mathsf{V}_G(d) \cap \mathsf{V}_G(d') = \{u\}$.

Because both d and d' belong to T_P, there exists a vertex v_i of the path P contained in both $\mathsf{V}_G(d)$ and $\mathsf{V}_G(d')$. Thus $v_i = u$, contradicting the assumption that P avoids u. □

E Analysis of ListConstruction

We present the proof of Lemma 3, describing the guarantees of algorithm ListConstruction. The parties run this algorithm locally to convert the input tree T into a list representation that provides a few special properties.

Lemma 3. *Consider a rooted tree T, and let v_{root} denote its root.*

Then, $\mathsf{ListConstruction}(T, v_{\mathsf{root}})$ returns, in finite time, a list of vertices L with the following properties:

1. *If $\big|\mathsf{V}(T)\big| > 1$, then for any $i < \big|L\big|$, the vertices L_i and L_{i+1} are adjacent in T.*
2. *The list L contains $\big|L\big| \leq 2 \cdot \big|\mathsf{V}(T)\big|$ elements, and, for every vertex $v \in \mathsf{V}(T)$, we have $L(v) \neq \varnothing$.*
3. *Consider a vertex $v \in \mathsf{V}(T)$, and let $i_{\min} = \min L(v)$ and $i_{\max} = \max L(v)$. Then, a vertex u is in the subtree rooted at v if and only if $L(u) \subseteq [i_{\min}, i_{\max}]$.*

[1] Such a maximal clique exists since every edge of G is contained in a block, and every block is a clique in a block graph.

4. *For any two vertices $v, v' \in \mathsf{V}(T)$ and any $i \in L(v)$ and $i' \in L(v')$, the lowest common ancestor of v and v' is in the set $\{L_k : \min(i, i') \leq k \leq \max(i, i')\}$.*

Proof. First, we prove that $\mathsf{ListConstruction}(T, v_{\mathsf{root}})$ terminates in a finite amount of time. The algorithm follows a depth-first search (DFS) traversal, which has a time complexity of $O(|V| + |E|)$ for a graph with vertex set V and edge set E. Since T is a tree, it has $|E| = |V| - 1$ edges, leading to an overall complexity of $O(|V|)$. Thus, the algorithm terminates within a finite number of steps. Then, we prove each property in order.

Property 1. This follows directly from the definition of DFS. At each step of the DFS traversal, if a vertex has an unvisited child, we move to that child. Otherwise, we backtrack to the parent. This ensures that each successive pair of vertices in L represents either a move to a child or a return to a parent, both of which are adjacent in T.

Property 2. We first show that $\left|L\right| \leq 2 \cdot \left|\mathsf{V}(T)\right|$ holds. We proceed by induction on the number of vertices $m := |\mathsf{V}(T)|$.

Base case: If T consists of a single vertex, then $L = [v_{\mathsf{root}}]$. Since $\left|L\right| = 1 \leq 2 \cdot |\mathsf{V}(T)| = 2$, the claim holds.

Inductive step: Suppose the claim holds for all trees with at most $m - 1$ vertices. Consider a tree T with m vertices. Let T' be the tree obtained by removing a leaf v and its incident edge (u, v).

By the induction hypothesis, the DFS traversal on T' results in a list L' of length at most $2(m - 1)$. When we add v back, the DFS first visits v once and then backtracks to u, adding two elements to L'. Therefore, $|L| = |L'| + 2 \leq 2(m - 1) + 2 = 2m$, which completes the induction.

To show that every vertex appears in L, note that the DFS visits each vertex at least once upon first exploration, ensuring that every vertex is included.

Property 3. During DFS, the first occurrence of v in L happens before visiting any of its descendants. The last occurrence happens after all descendants have been visited. Hence, the indices corresponding to v enclose all its descendants in L. If u is in the subtree of v, all its occurrences in L appear between $i_{\min}$ and $i_{\max}$. Conversely, if $L(u) \subseteq [i_{\min}, i_{\max}]$, then DFS must have discovered u while visiting v's subtree, confirming that u is in the subtree rooted at v.

Property 4. If one of v or v' is an ancestor of the other, then the lowest common ancestor (lca) of two nodes v and v' is trivially in this range.

Otherwise, let $w := \mathsf{lca}(v, v')$. If $w \notin L_k : \min(i, i') \leq k \leq \max(i, i')$, then DFS would have visited v's subtree and then v''s subtree without encountering w. However, this contradicts the DFS property that it does not leave a subtree before visiting all its nodes. Thus, w must appear in the interval, completing the proof. □